
NiCE Voice Grid Media Worker Administration Guide Version 21.0.3.5

- [Copyright](#)
- [Introduction](#)
 - [Deployment Infrastructure](#)
 - [Operational Modes](#)
 - [TCP/IP Port Usage](#)
- [Installation and Setup](#)
 - [Before You Start](#)
 - [License Management](#)
 - [Overview](#)
 - [Windows-Specific Overview](#)
 - [Linux-specific Overview](#)
 - [RabbitMQ Pre-Upgrade Checks \(Windows & Linux\)](#)
 - [Basic Installation / Upgrade \(Windows\)](#)
 - [Basic Installation / Upgrade \(Linux\)](#)
 - [Prerequisites](#)
 - [Linux-Based Voice Grid Installation](#)
 - [Linux-Based Voice Grid Upgrade](#)
 - [RabbitMQ on a Linux System](#)
 - [Linux Based RabbitMQ Installation](#)
 - [Miscellaneous RabbitMQ Troubleshooting Tips \(Linux\)](#)
 - [Securing RabbitMQ Data](#)
 - [Securing RabbitMQ Mnesia on Windows](#)
 - [Securing RabbitMQ Mnesia on Linux](#)
 - [RabbitMQ TLS Configuration](#)
 - [Certificates Required for RabbitMQ TLS](#)
 - [Steps to Configure RabbitMQ TLS](#)
 - [Validation and Troubleshooting](#)
- [Model Management](#)
 - [Workflow Model Management](#)
 - [Installing new Workflow Models](#)
 - [Removing Workflow Models](#)

- [Call Summarization Model Management](#)
- [Configuration Items](#)
 - [System Level Settings](#)
 - [General Settings](#)
 - [Workflow Media Analysis Specific Settings](#)
 - [CallSummarization Specific Settings](#)
 - [ElevateAI Specific Settings](#)
 - [Transcription Specific Settings](#)
 - [Concurrency Control](#)
 - [GPU Use](#)
 - [Multiple Voice Grid Instances](#)
- [Monitoring and Health Check](#)
 - [Voice Grid Log Files](#)
 - [Additional CallSummarization Logs](#)
 - [HTTP Endpoints \(REST APIs\) for Monitoring and Checking Health](#)
 - [Get Health Information \(actuator/health\)](#)
 - [Get Resource Usage \(actuator/resourceUsage\)](#)
 - [Get Inventory](#)
 - [Restart Satellites \(restartSatellites\)](#)
- [Voice Grid in Call Summarization Implementation Mode \(Windows Only\)](#)
 - [Installation](#)
 - [Running the Call Summarization Grid](#)
 - [Troubleshooting](#)
- [Common Exception Protocol \(CEP\) Server Configuration](#)
 - [A Little About Certificates](#)
 - [A Little About Keystores](#)
 - [Using The Default CEP Client Certificate](#)
 - [Windows Instructions](#)
 - [Linux Instructions](#)
 - [Using Custom Certificates](#)
 - [Importing Custom Trusted Root Certification Authority Certificates](#)
 - [Importing a Dual-Use Certificate](#)
 - [Importing Encryption Certificates](#)
 - [Setting CEP Properties In Voice Grid](#)
 - [Mandatory Properties To Enable CEP](#)
 - [Optional CEP Properties](#)

Copyright

NiCE Voice Grid™ Administration Guide
Version 21.0.3

PROPRIETARY AND CONFIDENTIAL INFORMATION

Information herein is proprietary information and trade secrets of NiCE Ltd. and/or its affiliated companies (Affiliates). This document and the information herein is the exclusive property of NiCE and its Affiliates and shall not be disclosed, in whole or in part, to any third party or utilized for any purpose other than the express purpose for which it has been provided.

IMPORTANT NOTICE

Subject always to any existing terms and conditions agreed between you and NiCE or any Affiliate with respect to the products which are the subject matter of this document, neither NiCE nor any of its Affiliates shall bear any responsibility or liability to a client or to any person or entity with respect to liability, loss or damage caused or alleged to be caused directly or indirectly by any product supplied or any reliance placed on the content of this document. This includes, but is not limited to, any interruption of service, loss of business or anticipatory profits or consequential damage resulting from the use or operation of any products supplied or the content of this document. Information in this document is subject to change without notice and does not represent a commitment on the part of NiCE or any Affiliate.

All information included in this document, such as text, graphics, photos, logos and images, is the exclusive property of NiCE or in Affiliate and is protected by United States and international copyright laws. Permission is granted to use, view and photocopy (or print) materials from this document only in connection with the products to which this document relates and subject to the terms of license applicable to such products. Any other use, copying, distribution, retransmission or modification of the information in this document without the express prior written permission of NiCE or an Affiliate is strictly prohibited. In the event of any permitted copying, redistribution or publication of copyrighted material, no changes in, or deletion of, author attribution, trademark legend or copyright notice shall be made.

Products supplied may be protected by one or more of the US patents listed at www.nice.com/company/legal/patents. For the full list of trademarks of NiCE and its Affiliates, visit www.nice.com/company/legal/nice-trademarks. All other marks used are the property of their respective proprietors.

All contents of this document are: Copyright © 2026 NiCE Ltd. All rights reserved.

For assistance, contact your local supplier or nearest NiCE Customer Service Center:

**EMEA Region (Europe,
Middle East, Africa)**

Tel: +972-9-775-3800
Fax: +972-9-775-3000
email: support@nice.com

**APAC Region (Asia/
Pacific)**

Tel: +852-8338-9818
Fax: +852-2802-1800
email:

North America

Tel: 1-800-663-5601
Fax: +201-356-2197
email:
na_sales@nice.com

International

Headquarters-Israel
Tel: +972-9-775-3100
Fax: +972-9-775-3070
email: info@nice.com

support.apac@nice.com

**The Americas Region
(North, Central, South
America)**

Tel: 1-800-6423-611

Fax: +720-264-4012

email:

support.americas@nice.com

Israel

Tel: 09-775-3333

Fax: 09-775-3000

email: support@nice.com

France

Tel: +33-(0)1-41-38-5000

Fax: +33-(0)1-41-38-5001

Hong-Kong

Tel: +852-2598-3838

Fax: +852-2802-1800

United Kingdom

Tel: +44-8707-22-4000

Fax: +44-8707-22-4500

Germany

Tel: +49-(0)-69-97177-0

Fax:

+49-(0)-69-97177-200

NiCE invites you to join the NiCE User Group (NUG).

Visit the NUG Website at www.niceusergroup.org; and follow the instructions.

All queries, comments, and suggestions are welcome! Please email: nicebooks@nice.com.

For more information about NiCE, visit www.nice.com

Introduction

NiCE Voice Grid is a new type of "single-purpose" Grid service that features a simple deployment scheme (and network topology).

Deployment Infrastructure

At a high level, its dependent infrastructure consists of just two parts:

- Shared Network Files - needed for default Workflow implementation. Repositories of various Workbench model files (e.g., redaction workflow models) are modeled as UNC based file path patterns. Although not strictly required, these common repositories can be shared by all Voice Grid instances in the cluster. Moreover, input media in client requests are specified as file URIs (<file://<host>/path>); so shared network storage with the client system is a prerequisite.
- RabbitMQ AMQP Messaging Broker - common for both implementations The request and response queues configured on the RMQ represent the public interface of the VG service. These queues do not need to be configured; they are created automatically by the Voice Grid service on first starting up. Client systems must have read/write access to these queues.

Operational Modes

There are four modes of implementation of Voice Grid. The default mode is Workflow MediaAnalysis mode. The other modes of implementation are CallSummarization, ElevateAI and Transcription.

- In MediaAnalysis mode, Voice Grid provides a thin layer around Workbench processing through workflows. Within Workbench, media files containing speech are analyzed and processed. This mode is primarily used for phrase recognition (also called speech to text or ASR) and sentiment analysis. Additionally, text files generated from *previously ASR'd media* may be passed in for sentiment analysis.
- In CallSummarization, text transcripts are analyzed by the Natural Language Processing Satellite and Text Analysis Satellite to summarize the concepts addressed in the call.
- ElevateAI mode is similar to the Media Analysis mode in general concepts, but is specifically targeted at providing output for NiCE's ElevateAI product offering.
- In Transcription mode, Voice Grid provides an interface to transcribe audio files into JSON-based transcriptions using the T11 engine from Workbench. This implementation is contained in a docker container that will not use the installation procedures outlined in this document.

TCP/IP Port Usage

This section lists default port assignments to support firewall configuration.

Component	Default Port(s)	Purpose
Voice Grid Media Worker	8080	HTTP (health/metrics)
Voice Grid Media Worker	20200	JMX (diagnostics, normally disabled)
RabbitMQ	5672	AMQP
RabbitMQ	15672	RabbitMQ Management Console
NLP Satellite	51200 to 51299	RPC
Text Analysis Satellite	51300 to 50399	RPC

Installation and Setup

Before You Start



Always refer to the README.txt file (distributed with Voice Grid) for any breaking changes, or any other changes to consider or procedures that must be followed prior to, during, or immediately after an installation or upgrade.

License Management

The licensing required in Voice Grid v2.0.0 and older is no longer required.

Overview

Voice Grid consists of a Java Runtime Environment (JRE), the Voice Grid (NxVgMediaWorker) service, and a RabbitMQ service, which in-turn requires Erlang. The JRE is not required nor installed for RabbitMQ-only servers.

Because of the different filesystem structures, the install-directory and data-directory vary between Windows and Linux. On Windows, there are fixed directories, but the drive letters are customizable, whereas on Linux, there are only fixed directories, however, relocation can be achieved by using Linux's symbolic link mechanism to point Voice Grid's fixed directories to other disk volumes.

Within this document, there are references to <install-directory>, <data-directory> and <NxVgMediaWorker_DATA_ROOT>. Those are defined differently in the Overview subsections below for each of the two operating systems, Windows and Linux. The subdirectories beneath those locations, however, are the same for either operating system.

Windows-Specific Overview

The InstallVG.ps1 PowerShell script should be used to install all the necessary components for Voice Grid:

- Java Runtime Environment (JRE)
- RabbitMQ Server (Erlang & RabbitMQ)
- Voice Grid

This script streamlines installation, eliminates many manual steps and opportunities for human error, and standardizes on installation location, data locations, and the Voice Grid client user for RabbitMQ. It has three installation types:

- **RabbitMQServer** installs Erlang and RabbitMQ Server, sets up the Windows environment, removes the default "guest" user, sets permissions for the RabbitMQ Service user, and configures RabbitMQ with the required settings, and admin user and client user accounts. By default, port 5672 is used, and therefore must not be used by any other application. The installer script also enables the rabbitmq_management plugin, which provides a browser-based management interface at <http://localhost:15672>
- **VGWorker** installs the JRE and Voice Grid, sets up the Windows environment, sets permissions for the Voice Grid Service user, configures Voice Grid client user account needed to access the RabbitMQ Server.
- **Combined** installs the JRE, Erlang, RabbitMQ Server, and Voice Grid.



The "Combined" installation type is typically not used, except possibly in a lab, POC, or development environment. The typical installation will have RabbitMQServer installed on one server, and VGWorker installed on one or more separate servers.

The InstallVG.ps1 script allows optionally setting the drive letter for the installation drive (on to which the components get installed) and the drive letter for the data drive (on to which the component data will be located). Prior to VG 4.0, Voice Grid's data, including models and workflows, was all kept within its installation location, but beginning with VG 4.0, all data, *except for the .properties files*, have been moved to the separate data location. When upgrading from pre-4.0 version to v4.0+, the InstallVG.ps1 script will migrate the data to the new location.

Later in this document you may find references to <install-directory>, <data-directory> and <NxVgMediaWorker_DATA_ROOT>. On Windows, those refer to the following locations:

- <install-directory> = <InstallDrive>:\Program Files\Nexidia\Voice Grid
- <data-directory> = <DataDrive>:\ProgramData\Nexidia\Voice Grid
- <NxVgMediaWorker_DATA_ROOT> = <data-directory>\NxVgMediaWorker

When upgrading, the previous installation's data and configuration is retained, including user accounts and passwords, and in the case of VGWorker, the previous installation's models and workflows will be retained, any custom settings made to the application.properties file will be migrated to the new version, and the previous logback-spring.xml file will be retained to preserve any customized logging settings.

On a Windows system, the RabbitMQ configuration and log files are NOT under the user's %APPDATA% directory like RabbitMQ would normally locate them by default. Instead, the InstallVG.ps1 script sets the RABBITMQ_BASE environment variable so that they will be in the <data-drive>:\ProgramData\VoiceGrid\RabbitMQ directory. Do not change that environment variable or attempt to move their location manually; only ever use the InstallVG.ps1 script to change the installation and/or data drive letters. The paths (other than the drive letter) are not modifiable.

Voice Grid's dependency on RabbitMQ as the messaging infrastructure piece is generic. It is perfectly allowable to install Voice Grid on a Windows system and have it talk to an RabbitMQ instance running on a Linux server (or verse versa). For more information on installing the RMQ service on a Linux system, see the RabbitMQ on a Non-Windows System section.

Voice Grid can also be configured to a pre-existing RabbitMQ server instance (preferably with dedicated service user and virtual host). It is not sensitive to Erlang/RMQ version variations. However, this is beyond the scope of this document. Contact NiCE support for further information on using a pre-existing RabbitMQ instance.

Linux-specific Overview

The Linux implementation of Voice Grid is currently considered experimental.

IMPORTANT LIMITATIONS:

- **The Call Summarization implementation mode is not currently supported on Linux.** To use the Call Summarization implementation, you must use the Windows version of Voice Grid.
- **There is no automated installation or upgrade of RabbitMQ on Linux.** (The RabbitMQ server need not be running under the same operating system as the Voice Grid server, however, so RabbitMQ could be installed on a Windows server using Voice Grid's InstallVG.ps1 script for Windows, and then run Voice Grid itself on Linux.)



Later in this document you may find references to <install-directory>, <data-directory> and

<NxVgMediaWorker_DATA_ROOT>. On Linux, those refer to the following locations by default:

- <install-directory> = `/opt/Nexidia/Voice Grid`
- <data-directory> = `/var/Nexidia/Voice Grid`
- <NxVgMediaWorker_DATA_ROOT> = `<data-directory>/NxVgMediaWorker`

(NxVgMediaWorker_DATA_ROOT can be set to a custom value if needed.)

RabbitMQ Pre-Upgrade Checks (Windows & Linux)

Before upgrading RabbitMQ it is highly recommended to verify that all *non-experimental* feature flags are enabled. The InstallVG.ps1 script enable all feature flags after installation and upgrades; however, some RabbitMQ plugins can make additional feature flags available. Therefore, if any plugins were manually enabled after installation, there could be feature flags that came into existence post installation/upgrade and were not enabled.



To guarantee upgradability, all non-experimental feature flags **must** be enabled.

Unfortunately, there is no way for the installer script to distinguish between normal flags that must be enabled and experimental flags that should be left disabled, and thus this recommendation to check.

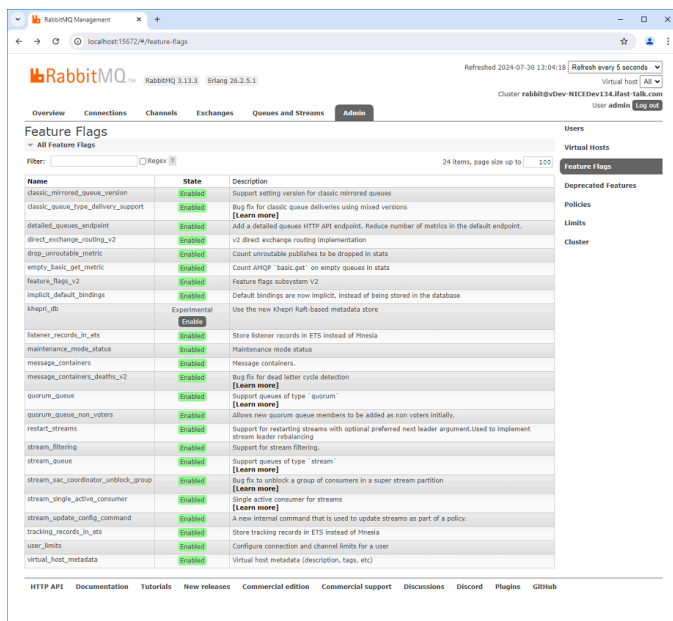


On Linux, since RabbitMQ installation is a manual process, nothing guarantees that feature flags were enabled previously and therefore this check is essential.

To check feature flag states, open the RabbitMQ management console in any browser *running on that RabbitMQ server* by going to <http://localhost:15672>, logging in as user admin with the admin password ("admin" by default if you never set one), clicking on the **Admin** tab at the top, then clicking on the **Feature Flags** tab at the right.

- If any flags that are **not** experimental are disabled, **enable** them.
- Do **not** enable flags marked as *experimental*.

Below is an example of what that screen may look like.(The list of available flags can vary by RabbitMQ version.):



If there are any **non-experimental** feature flags that are not enabled and the management console doesn't allow you to enable them, you can enable them from a command prompt.

- **Windows:** Open a "cmd" window, navigate (cd) to the `<install-directory>\RabbitMQ\rabbitmq_server-<version>\sbin` folder, then run the command:



```
rabbitmqctl.bat enable_feature_flag all
```

- **Linux:** Open a bash shell window, then run the command:

```
sudo rabbitmqctl enable_feature_flag all
```

Then double-check again in the RabbitMQ management console that all non-experimental flags are being reported as enabled.

Basic Installation / Upgrade (Windows)



When upgrading, before running the InstallVG.ps1 script, you should first quiesce Voice Grid and stop the RabbitMQ service, on whatever server it is running on. (If RabbitMQ runs on the same server you are upgrading, the script will detect that RabbitMQ is running and not allow you to proceed until it has been stopped; however, if you are upgrading a different Voice Grid server, it cannot check, so stopping RabbitMQ should *always* be done manually before beginning any upgrade.)

The InstallVG.ps1 script must be run as Administrator. Launch PowerShell as administrator and then run the script from that PowerShell window.

The script's default behavior requires that the various components and installation tools used by the InstallVG.ps1 script be **co-located** with the script (in the same directory as the script):

- The JRE with one of these three naming conventions (not required for RabbitMQ-only installs):
 - jre-<version>-win.zip (example: jre-17.56.16-win.zip) **OR**
 - zulu-jre<version>-win_x64.zip (example: zulu-jre-17.56.16-win_x64.zip) **OR**
 - zulu<version>-win_x64.zip (examples: zulu17.56.16-sa-jre17.0.14-win_x64.zip)
- Carbon-<version>.zip (example: Carbon-2.15.1.zip) – ALWAYS REQUIRED
- Erlang with one of these two naming conventions - Required for RabbitMQServer or Combined install types:
 - otp-<version>-win64.exe (example: otp-26.2.5.6-win64.exe) **OR**
 - otp_win64_<version>.exe (example: otp_win64_26.2.5.6.exe)
- rabbitmq-server-<version>.exe (example rabbitmq-server-3.13.7.exe) – Required for RabbitMQServer or Combined install types.
- NxVgMediaworker-<version>.zip (example: NxVgMediaworker-20.0.0.2.zip) – Required for VGWorker or Combined install types.

These may be individual files **OR** contained within a single zip file named NxVgInstallerBundle-<version>.zip.

- If multiple NxVgInstallerBundle-<version>.zip files are found, that is considered an error.
- If a single NxVgInstallerBundle-<version>.zip file is found, it will be used and the items above will be expected to be contained within it.
- If no NxVgInstallerBundle-<version>.zip file is found, it will expect the individual files.

This default behavior can, however, be overridden by using the -InstallFrom option of the InstallVG.ps1 script. With that option, the 5 items above will still need to be in one place, but that location can be separate from the script, and could even be a UNC path or zip file that contains the 5 items.

 The default means of distributing Voice Grid is to bundle the five (5) components, mentioned above, into a single zip file named `NxVgInstallerBundle-<version>.zip` and distribute that zip and the InstallVG.ps1 script. In this case, the script's `-InstallFrom` option must be used script to provide the path (absolute or relative to the script) and name of the bundle zip file.

The basic usage of the script is as follows:

```
InstallVG.ps1 -Action <Install|Uninstall|Report> -InstallType <VGWorker|RabbitMQServer|Combined> -
ServiceUser <username> -ServicePassword <password> [Optional Parameters]
```

- Where `-Action Install` should be used for installing or upgrading.
- And where `-ServiceUser` and `-ServicePassword` are required for `-Action Install`, and will be ignored for `Uninstall`. This specifies a Windows user account to be used for the service(s) installed (Voice Grid and/or RabbitMQ). If the user is a domain user account, the domain name must be included. For example,

```
-ServiceUser yourdomain\vgserviceuser -ServicePassword iamthepassword
```



Do not use any of the built-in users, such as Local Service or Network Service. Use an actual user account, which the services will run under.

- And where the most commonly used [Optional Parameters] are:
 - `-InstallFrom <location>`: Specifies path to the components or the path\name of a zip that bundles all components. In both cases, the path can be relative to the location of the InstallVG.ps1 script, or absolute, including a UNC path.
 - `-VGClientPassword <password>`: Specifies the client password Voice Grid uses to connect as a client to RabbitMQ for the "vgclient" user. If omitted, the password "nxVGc13nt" will be used if installing, or the previous password will be retained if upgrading.
 - `-InstallDrive <drive-letter|\*>`: Specify as * to indicate that an upgrade should retain its current installation location.
 - `-DataDrive <drive-letter|\*>`: Specify as * to indicate that an upgrade should retain its current data location.
 - `-VGImpl <MediaAnalysis|CallSummarization|ElevateAI>`: Set's installation's implementation mode. Applies only during installation (ignored for upgrades) and if omitted, the default is MediaAnalysis.



The "Combined" -InstallType is not recommended for MediaAnalysis because MediaAnalysis may consume all available resources, possibly choking out RabbitMQ. It is possible to configure the number of threads to a low enough value to allow Voice Grid and RabbitMQ to co-exist, but that impacts performance. (See the [workflow.processing.concurrency](#) property in the [Concurrency Control](#) section.) For MediaAnalysis installations, ideally, Voice Grid and RabbitMQ should be on separate servers.



Run “InstallVG.ps1 -Help” to get much more detailed information about all parameters.

After installation, Voice Grid will have the following directory structure:

Directory	Purpose
<install-directory>\zulu-jre	This is the root folder of the Java Runtime Environment
<install-directory>\NxVgMediaWorker	This is the root folder for the Voice Grid server.
<install-directory>\NxVgMediaWorker\application.properties	This is the configuration file for the application. For more information on configuration options, see Configuration Items .
<install-directory>\NxVgMediaWorker\logback-spring.xml	This is the configuration file for logback. The rolling policy for logs can be customized based on client installation needs.
<install-directory>\NxVgMediaWorker\bin	Contains the service installation script.
<install-directory>\NxVgMediaWorker\doc	Contains documentation relevant to Voice Grid.
<install-directory>\NxVgMediaWorker\jniLibs	Contains native Workbench library files.
<install-directory>\NxVgMediaWorker\libs	Contains the Java service library file.
<install-directory>\RabbitMQ	This is the root folder of the RabbitMQ server, if RabbitMQ is installed.
<install-directory>\Erlang OTP	This is the root folder of the Erlang component, present only if RabbitMQ is installed.

<data-directory>\NxVgMediaWorker>	This is the root folder for the Voice Grid server's data and is synonymous with <NxVgMediaWorker_DATA_ROOT>.
<NxVgMediaWorker_DATA_ROOT>\models\workflow	This is the destination directory for workflow models.
<NxVgMediaWorker_DATA_ROOT>\logs	This is where Voice Grid's log files will reside.
<data-directory>\RabbitMQ	This is the root folder of RabbitMQ's data, if RabbitMQ is installed.

 For a VGWorker (or Combined) installation, if the model provided for Voice Grid is a zip model — usually has an extension like *.nxzm.zip — then it needs to be placed *without unzipping it*. Once a model is placed it can be validated by running the inventory call shown in [Get Inventory](#) which should return the GUID.

Basic Installation / Upgrade (Linux)

Prerequisites

If not already installed in the Linux distribution you're using, you must install the dos2unix and ca-certificates packages prior to installing Voice Grid.:

```
$ sudo yum install dos2unix
$ sudo yum install ca-certificates
```

Additionally, libxml2 is also required, but is usually already included in Red Hat distributions, so installing it may not be necessary.

```
$ sudo you install libxml2
```

 If you do not install the required packages before attempting to install Voice Grid, the Voice Grid installer will check to see if they're installed, and if not, you will have the opportunity to install them at that point.

If media files are located on a Windows share, the Linux server must be configured such that any Windows UNC paths (\\server\share) must be mounted as CIFS type mounts in Linux, mounting to those Windows shares with a mount path of /mnt/server/share, where the server name must be all lower-case, and the share name must match the case in Windows. Details on how to do that are beyond the scope of this document, but generally such mount points go into the /etc/fstab file. Refer to Linux documentation for details.

Linux-Based Voice Grid Installation

Linux support was introduced in Voice Grid version 2.0.0, and installation greatly simplified in Voice Grid version 20.1.0. At this time, however, Linux support is still considered experimental, and the Call Summarization implementation mode is not supported on Linux.

Compared with the mainstay Windows deployment model, the Linux standard operating procedure for setting up a Voice Grid site is characterized by a few OS specific features/limitations:

1. Voice Grid is distributed as a standard rpm package and installed with `yum`
2. Voice Grid includes the recommended Java Runtime Environment (JRE), which can be overridden if necessary. (Override instructions are below.)
3. Voice Grid service managed by the standard Linux `systemd` utility
4. There is a fixed installation directory `/opt/Nexidia/VoiceGrid`
5. The default data directory is `/var/Nexidia/VoiceGrid`, which can be overridden if necessary. (Override instructions are below.)
6. A special service user (`nexidia`) must be provisioned with sudo privilege before installing Voice Grid

The art of Unix/Linux user administration is out of the scope of this document, but we can add the required service user with

```
$ sudo useradd nexidia -c "VG Service User"
```

Then set its password with

```
$ sudo passwd nexidia
```

And finally, give it sudo privilege with

```
$ sudo usermod -aG wheel nexidia
```



The following installation procedures are conducted after you have logged in as (or su to) the nexidia user and have copied the RPM file to somewhere on the Linux server, or accessible via a mount point on it, which we'll just refer to as `<path-to-rpm>`. Substitute your actual path.

Where you see `<version>`, below, substitute the actual version within your RPM file's name.

Step 1. Install the NxVgMediaWorker Package:

```
$ sudo yum install <path-to-rpm>/NxVgMediaWorker-<version>.noarch.rpm
```

The directory structure is revealed with the rpm tool (-ql=Query List):

```
$ rpm -ql NxVgMediaWorker-<version>
/etc/systemd/system/NxVgMediaWorker.service
/opt/Nexidia
/opt/Nexidia/VoiceGrid
/opt/Nexidia/VoiceGrid/NxVgMediaWorker
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/README.txt
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/application.properties
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/bin
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/bin/NxVgMediaWorker.service
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/bin/NxVgMediaWorker.sh
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/doc
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/doc/NiCE Voice Grid Media Worker Administration Guide.html
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/doc/NiCE Voice Grid Media Worker Programmer's Guide.html
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/doc/NiCE Voice Grid Media Worker System Requirements.html
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/doc/Nexidia.Workbench_ThirdPartyLicenses.txt
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/doc/ThirdPartyLicenses.txt
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/doc/ThirdPartyLicenses_Addendum.txt
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs/libNexidia.Workbench.JNI.so
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs/libavcodec.so.57
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs/libavformat.so.57
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs/libavutil.so.55
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs/libonnxruntime_providers_cuda.so
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs/libonnxruntime_providers_shared.so
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs/nexidia-workbench.jar
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/libs
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/libs/NxVgMediaWorker-<version>.jar
/opt/Nexidia/VoiceGrid/NxVgMediaWorker/logback-spring.xml
/opt/Nexidia/VoiceGrid/zulu-jre (and content below, dependent upon specific JRE version)
/var/Nexidia
/var/Nexidia/VoiceGrid
/var/Nexidia/VoiceGrid/NxVgMediaWorker
/var/Nexidia/VoiceGrid/NxVgMediaWorker/logs
/var/Nexidia/VoiceGrid/NxVgMediaWorker/models
/var/Nexidia/VoiceGrid/NxVgMediaWorker/models/workflow
```

Step 2: **OPTIONAL** - Establish a <data-directory>

As of Voice Grid 4.0, data files have been separated from installation / application files. Data files include things such as workflows and logs. By default, the <data-directory> root is **/var/Nexidia/VoiceGrid**, which gets created automatically installation time, as of Voice Grid 20.1.0.



ONLY if this needs to be different, you may either...

- Simply create a symlink at that location to point to the physical location that you want to use (simplest),
OR

- Follow the steps below to override the location by setting a **NxVgMediaWorker_DATA_ROOT** environment variable.

The <data-directory> can be anywhere *outside* the <install-directory>. This is beneficial, because after moving to this new, separate data directory, upgrades will be much simpler because they won't risk loss of data files since they will be kept outside the <install-directory>. Once you've determined the name & location of the <data directory>, you must:

- Create the <data-directory> (substitute your specific location for "<data-directory>") and set its ownership to user nexidia, group nexidia:

```
$ sudo mkdir -f <data-directory>
$ sudo chown nexidia:nexidia <data-directory>
```

- Edit the /opt/Nexidia/VoiceGrid/environment file and set NxVgMediaWorker_DATA_ROOT to your <data-directory>



If the file does not exist, create it, and then use chown to set owner:group, as in:

```
$ sudo chown nexidia:nexidia /opt/Nexidia/VoiceGrid/environment
```

```
$ sudo vi /opt/Nexidia/VoiceGrid/environment
```

Use SHIFT+A or SHIFT+I to enter insert mode, and append to the file a line that contains the following (substituting your specific location for "<data-directory>")

```
NxVgMediaWorker_DATA_ROOT=<data-directory>
```

OR if you wish to mirror Voice Grid's default subdirectory structure,

```
NxVgMediaWorker_DATA_ROOT=<data-directory>/NxVgMediaWorker
```

For example, enter `NxVgMediaWorker_DATA_ROOT=/var/MyCustomDirectory`, then save the file
(typically ESC w q)

- Create required subdirectories under `<NxVgMediaWorker_DATA_ROOT>` (substituting your specific location for whatever you set "`NxVgMediaWorker_DATA_ROOT`" to) and set their ownership to user `nexidia`, group `nexidia`:

```
$ sudo mkdir -p <NxVgMediaWorker_DATA_ROOT>/logs
$ sudo chown nexidia:nexidia <NxVgMediaWorker_DATA_ROOT>/logs

$ sudo mkdir -p <NxVgMediaWorker_DATA_ROOT>/models/workflow
$ sudo chown nexidia:nexidia <NxVgMediaWorker_DATA_ROOT>/models
$ sudo chown nexidia:nexidia <NxVgMediaWorker_DATA_ROOT>/models/workflow
```

Step 3: **OPTIONAL** - Using your own Java Runtime Engine

Voice Grid includes a robust JRE from Azul. It is recommended to use this JRE, as this is the JRE that we have validated against. However, we recognize that some businesses have policies that require them to install and maintain a different JRE, i.e., from Oracle.



Voice Grid will **not** use the system value for `JAVA_HOME` if it is set. This is done to prevent Voice Grid from unintentionally using the wrong JRE.

ONLY if you need to use (and take on the responsibility of maintaining) your own JRE, you may override the included JRE following the steps below, but *please refer to the Voice Grid System Requirements document regarding the Java version that Voice Grid requires.*

- Edit the `/opt/Nexidia/VoiceGrid/environment` file and set `JAVA_HOME` to where your JRE is located. For example, the line may be



If the file does not exist, create it, and then use `chown` to set owner:group, as in:

```
$ sudo chown nexidia:nexidia /opt/Nexidia/VoiceGrid/environment
```

```
$ sudo vi /opt/Nexidia/VoiceGrid/environment
```

Use `SHIFT+A` or `SHIFT+I` to enter insert mode, and append to the file a line that contains the following
(substituting your specific location for "`<jre-directory>`")

JAVA_HOME=<jre-directory>

For example, enter `JAVA_HOME=/usr/bin/java`, then save the file (typically ESC w q)

Step 4. Configuration and Sanity Check

```
$ cat /opt/Nexidia/VoiceGrid/NxVgMediaWorker/application.properties
spring.rabbitmq.addresses=localhost:5672
spring.rabbitmq.username=vgclient
spring.rabbitmq.password=nxVGc13nt
```

Amongst other content in the file, the above three properties must be set. The values shown above are default values that will be present on a fresh installation. If you upgraded, then values set prior to the upgrade should have been migrated as well as any additional property values that were previously set. **Please verify their values.**



The `spring.rabbitmq.username` property **must** be set to `vgclient`.

Note as long as the network connectivity exists, the RabbitMQ instance does not have to be running on this host or another Linux host. The messaging communication protocol is platform neutral.



However, at present, any Linux installation of RabbitMQ is completely manual. To install RabbitMQ on Windows, you may use the Windows version of Voice Grid's installation bundle and the `InstallVG.ps1` PowerShell script to perform an easy, automated installation of only RabbitMQ using the `"-InstallType RabbitMQServer"` command-line option.

For a fresh installation, you will need to set the `spring.rabbitmq.password` value to the one that was set during the RabbitMQ installation, and you must set the `spring.rabbitmq.address` value to point to the server on which RabbitMQ was installed. Update `/opt/Nexidia/VoiceGrid/NxVgMediaWorker/application.properties`

- Using `vi` or any other text editor to edit `/opt/Nexidia/VoiceGrid/NxVgMediaWorker/application.properties`
- Set the values of `spring.rabbitmq.address` (if not running on the same server) and `spring.rabbitmq.password` as follows:

```
spring.rabbitmq.address=<IpOrDnsNameOfServerRunningRabbitMQ>:<Port>
spring.rabbitmq.username=vgclient
spring.rabbitmq.password=<WhateverPasswordYouSpecifiedDuringTheRabbitMQInstallation>
```



The default port of 5672 should be specified unless you customized your RabbitMQ installation to use a different port, in which case, specify that port number.

- Save the file.

The following command displays the VG service configuration:

```
$ sudo cat /etc/systemd/system/NxVgMediaWorker.service
[Unit]
Description=NiCE Voice Grid Media Worker Service
After=syslog.target

[Service]
Environment=LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/opt/Nexidia/VoiceGrid/NxVgMediaWorker/jniLibs
User=nexidia
WorkingDirectory=/opt/Nexidia/VoiceGrid/NxVgMediaWorker
ExecStart=/opt/Nexidia/VoiceGrid/NxVgMediaWorker/bin/NxVgMediaWorker.sh
Restart=on-failure
RestartSec=120
TimeoutStopSec=1200

[Install]
WantedBy=multi-user.target
```

Step 5. Administrate the Systemd Service

For this step, we are using the standard Linux tools `systemctl`.

First, whenever making a change to the `.service` file, we need to run the following command to refresh the service configuration:

```
| sudo systemctl daemon-reload
```

Check the running status of the VG service:

```
| sudo systemctl status NxVgMediaWorker
```

To start the service:

```
| sudo systemctl start NxVgMediaWorker
```

To stop the service:

```
| sudo systemctl stop NxVgMediaWorker
```

Additionally, the `journalctl` utility, also a part of the `systemd` system, can also be employed to continuously monitor the output and logging status of Voice Grid.

Linux-Based Voice Grid Upgrade

To upgrade Voice Grid on Linux, the following steps must be performed:

1. Stop the NiCE Voice Grid Media Worker Service.
2. If upgrading from a version newer than 20.0.0, and you had customized Voice Grid's logging (unlikely), back up the logback-spring.xml file for later reference (the update will overwrite this file).
3. If upgrading from a version older than 4.0, back-up the files under the old installation's `<install-directory>/models/workflow` directory
4. Update Voice Grid.

```
$ sudo yum update <path-to-rpm>/NxVgMediaWorker-<version>.noarch.rpm"
```

If you ever need to force a re-installation of the *same* version, you may *substitute* the following for "yum update":



```
rpm -U -ivh --force
```

However, never use that to force a downgrade. Downgrading Voice Grid is not supported.

6. Reference the backed-up application.properties file for settings that must be configured in the new Voice Grid. The update will have replaced any custom settings, so it is critical to set them in the new files based up on the old files.
7. If upgrading from a version newer than 20.1.0, and you had customized Voice Grid's logging (unlikely), reference the backed-up logback-spring.xml file and add your prior customizations to the new logback-spring.xml file.
8. If upgrading from a version older than 20.1.0, you will need to...
 - a. BEFORE UPGRADE: Back up wherever the workflows are located in the old version.
 - b. AFTER UPGRADE: Copy the workflows that you backed up into `<NxVgMediaWorker_DATA_ROOT>/models/workflows`. (This is typically `/var/Nexidia/VoiceGrid/NxVgMediaWorker/models/workflows` unless NxVgMediaWorker_DATA_ROOT has been customized per the optional step in the [Linux-Based Voice Grid Installation](#) section.)
9. Start NiCE Voice Grid Media Worker Service.

RabbitMQ on a Linux System



This is still experimental, and there is no automated process to install or configure RabbitMQ on Linux. Using Windows is highly recommended until such time running it on Linux has been fully vetted and documented. You may try this, but getting RabbitMQ working can be tricky, and avoiding significant issues when upgrading RabbitMQ can be even trickier. If you use RabbitMQ on Linux, you are entering uncharted waters and **NiCE will not be able to provide support for any RabbitMQ-related issues**.

Linux Based RabbitMQ Installation

If upgrading RabbitMQ, please review the [RabbitMQ Pre-Upgrade Checks \(Windows & Linux\)](#) section before proceeding.

See <https://www.rabbitmq.com/install-rpm.html> for information on installing RabbitMQ on Linux. This will include installing the Erlang and setting the ERLANG_HOME environment variable before installing RabbitMQ.



Verify ERLANG_HOME is properly set after installing Erlang, and BEFORE installing RabbitMQ.

After installing RabbitMQ, you will probably want to establish a location for its data that is not tied to the service's user. Refer to RabbitMQ's website for more information about the use of RABBITMQ_BASE.

After installing RabbitMQ, you will need to start the service as indicated by the RabbitMQ website, then do some basic configuration. The scripts needed to do this should have been added to your path by the rabbitmq installer.

Enable all feature flags:

```
| sudo rabbitmqctl enable_feature_flag all
```

Add an admin user (this will be handy when using the browser-based management console):

```
| sudo rabbitmqctl add_user admin <WhateverPasswordYouWantAdminToHave>
```

OR if it is an upgrade, and you want to change an existing 'admin' user account password, use:

```
| sudo rabbitmqctl change_password admin <WhateverNewPasswordYouWantAdminToHave>
```

Enable administrator privileges for user admin

```
| sudo rabbitmqctl set_user_tags admin administrator
```

Grant user admin full access to virtual host

```
| sudo rabbitmqctl set_permissions admin ".*" ".*" ".*"
```

Remove default "guest" account for security reasons:

```
| sudo rabbitmqctl delete_user guest
```

Add the vgclient user (Note this user's password must match the password provided in Voice Grid's application.properties file.):

```
| sudo rabbitmqctl add_user vgclient <WhateverPasswordYouWant>
```

OR if it is an upgrade, and you want to change an existing 'admin' user account password, use:

```
| sudo rabbitmqctl change_password vgclient <WhateverNewPasswordYouWant>
```

Set permissions for the vgclient user:

```
| sudo rabbitmqctl set_permissions vgclient ".*" ".*" ".*"
```

Add the mediaworker user:

```
| sudo rabbitmqctl add_user mediaworker !mediaworker
```

Set permissions for the mediaworker user:

```
| sudo rabbitmqctl set_permissions mediaworker ".*" ".*" ".*"
```

Add the virtual host for Voice Grid:

```
| sudo rabbitmqctl add_vhost vg
```

Set permissions for the admin, vgclient and mediaworker users on the virtual host:

```
| sudo rabbitmqctl set_permissions admin --vhost vg ".*" ".*" ".*"  
| sudo rabbitmqctl set_permissions vgclient --vhost vg ".*" ".*" ".*"  
| sudo rabbitmqctl set_permissions mediaworker --vhost vg ".*" ".*" ".*"
```

Enable the browser-based management console at <http://localhost:15672/>:

```
| sudo rabbitmq-plugins enable rabbitmq_management
```

Miscellaneous RabbitMQ Troubleshooting Tips (Linux)

- Installation must be done using sudo to avoid permission related complications.
- Make sure ERLANG_HOME was set before attempting to run any of the rabbitmq cli commands.

A Potential Issue Caused by Erlang Cookie Mismatch. If running into authentication errors during the installation step, with the following signature messages:

- ```
| *CLI tool fails to authenticate with the server (e.g. due to CLI tool's Erlang cookie not matching that of the server)
| *Authentication failed (rejected by the remote node), please check the Erlang cookie
```

A quick fix is to copy the system erlang cookie (<the installer user's home directory>\.erlang.cookie) to the service user's home directory.

---

## Securing RabbitMQ Data

By default, RabbitMQ stores the content of its queues — its mnesia, as it calls it — unencrypted in the filesystem of the server on which RabbitMQ is installed. Depending upon what Voice Grid is being used for, the data in the queues could contain PII (Personal Identifiable Information). Storage is typically very short-lived, but this does create a security concern if processing PII. The location where RabbitMQ stores that data, however, can be customized, allowing it to be placed on a separate, encrypted hard drive or NAS, or a separate server that has secure storage.

---

## Securing RabbitMQ Mnesia on Windows

Establishing a separate location for RabbitMQ's mnesia is simple, and can be done at installation time, or moved later. The InstallVG.ps1 script can be used to set that location to any local or network (UNC) path.

- To set the mnesia location at installation time, simply provide the script with the optional parameter, `-RMQSecureRoot <PathToSecureStorageLocation>` and RabbitMQ's mnesia location will be established within a directory under the provided <PathToSecureStorageLocation>.
- To change the mnesia location after installation or during an upgrade, simply run the script, specifying the optional parameter, `-RMQSecureRoot <PathToSecureStorageLocation>` and the script will make a backup of the mnesia in its current location, and then copy it to the new location, under the provided <PathToSecureStorageLocation>.



When changing the location post-installation, *after verifying Voice Grid is functioning with the new location*, you should delete the backup it made so that any PII data that was in the old location isn't left sitting around. The location of the backup will be displayed by the InstallVG.ps1 script.

---

The <PathToSecureStorageLocation> can be any valid local or UNC path.

Examples:

```
-RMQSecureRoot R:\AnyFolder
```

or

```
-RMQSecureRoot \\AnotherServer\ShareName
```



If the path is a UNC path, on the remote server:



1. The *user running the script* must have full access permissions to the specified `<PathToStorageLocation>` in order for the installation script to create the necessary subdirectories and grant the necessary permissions on them for the service user.
2. The Windows **share** itself, not just the folder being shared, must grant at least "read" and "change" permissions to the service user, otherwise whatever permissions the script sets for the mnesia folders will be reduced to the permissions of the share and RabbitMQ will fail to run.

.....

The "effective access" of the share can be checked using Windows Explorer from the RabbitMQ server to view that share's properties:

.....

1. Right-click on share and choose **ScalingProperties**.
2. Click on the **Security** tab at the top of the Properties windows.
3. Click on the **Advanced** button near the bottom of the Window.
4. Click on the **Effective Access** tab of the new window that opened.
5. Click on **Select a user**, and choose the service user.
6. Click on the **View Effective Access** button.

.....

When using a UNC path, RabbitMQ will not be able to monitor free space. It will log a warning in its log file every 2 minutes, stating that it is expecting a path to "start with a drive letter like C:", and then after 6 hours it will log an error stating, "Free disk space monitor failed to start!" and the warnings will stop. This in and of itself is not a problem, but it does mean that the UNC path for RabbitMQ's mnesia must have sufficient free space.



Though normally the mnesia is very transient and therefore does not require much space, if, for example, the RabbitMQ service was left running, and while the all Voice Grid services were stopped, the client was still producing requests for Voice Grid, OR if the client was stopped after sending a lot of work to Voice Grid and Voice Grid was sending responses back while the client was stopped, that could consume considerable disk space.

For whatever path is specified, a RabbitMQ folder will be created in that location, and the mnesia will be stored under that folder.



If there is a need to move from a location previously specified by `-RMQSecureRoot` to the normal data location that the mnesia would reside if `-RMQSecureRoot` had never been specified, you may use

`-RMQSecureRoot OFF` (where OFF must be all upper-case).

Run `InstallVG.ps1 -Help` to get much more detailed information about all parameters.

---

## Securing RabbitMQ Mnesia on Linux

Securing this data on Linux has not been tested and therefore is not documented here. RabbitMQ's website should provide sufficient information to do this, but be aware that certain data under RabbitMQ's data directory will need to be moved to the secure location. Starting RabbitMQ without moving that data will cause RabbitMQ to start fresh, potentially causing loss of data if RabbitMQ had already been being used, including any work that was in-flight when stopped to upgrade the software. For more information, refer to RabbitMQ's website about the use of RABBITMQ\_MNESIA\_BASE.

---

## RabbitMQ TLS Configuration

Enabling TLS for RabbitMQ provides encryption for data in transit between Voice Grid and RabbitMQ.

This requires setting up RabbitMQ to use TLS and configuring Voice Grid to connect to RabbitMQ using TLS. The steps to configure RabbitMQ TLS are documented in the RabbitMQ TLS guide: <https://www.rabbitmq.com/ssl.html>

The certificate generation can be done using OpenSSL or any other certificate authority. Certificates must be in PEM format. The certificates must be trusted by both RabbitMQ and Voice Grid.



The details of actual certificate generation and management are outside the scope of this document.

---

## Certificates Required for RabbitMQ TLS

- CA certificate (ca\_cert.pem)
- Server certificate (server\_cert.pem)
- Server private key (server\_key.pem)
- Client certificate (client\_certificate.pem) if using client certificate authentication
- Client private key (client\_key.pem) if using client certificate authentication

Though each of those certificates are required, the exact file names above are not required, but are used in the example configuration, below.

---

## Steps to Configure RabbitMQ TLS

- Generate or obtain SSL/TLS certificates for RabbitMQ server and Voice Grid client.
- Configure RabbitMQ to use the server certificate and enable TLS on the desired port (default is 5671 for TLS).
- Update Voice Grid's `application.properties` file to enable TLS and provide the client certificate details.
- Restart RabbitMQ and Voice Grid services to apply the changes.

RabbitMQ TLS configuration requires the following properties to be set in `<data-directory>\RabbitMQ\rabbitmq.conf`:

```
listeners.ssl.default = 5671
ssl_options.cacertfile = /path/to/ca_certificate.pem
ssl_options.certfile = /path/to/server_certificate.pem
ssl_options.keyfile = /path/to/server_key.pem
ssl_options.verify = verify_none
ssl_options.fail_if_no_peer_cert = false
```

A typical configuration in Voice Grid's `<install-directory>\NxVgMediaWorker\application.properties` file for a TLS connection to RabbitMQ would include:

```
spring.rabbitmq.addresses=your_rabbitmq_server:5671
spring.rabbitmq.username=vgclient
spring.rabbitmq.password=your_password
spring.rabbitmq.ssl.enabled=true
spring.rabbitmq.ssl.verify-hostname=false
spring.rabbitmq.ssl.algorithm=TLSv1.3
spring.rabbitmq.ssl.validate-server-certificate=true
spring.rabbitmq.ssl.trust-store-type=PKCS12
spring.rabbitmq.ssl.trust-store=path/to/trust_store
spring.rabbitmq.ssl.trust-store-password=truststore_password
```

---

## Validation and Troubleshooting

After installation, check the health and inventory endpoint from a browser to validate the service is running properly. See relevant sections for details.

Most installation and service start problems are caused by environmental factors. Examine the generated Installation Log (NxVgMediaWorker\_PR.\*) and Service Log (NxVgMediaWorker.\*) for clues.



After an upgrade from a VG v1.x installation, if the `application.properties` file includes a line setting `redaction.processing.concurrency`, then the service will look like it is running, but will not actually be running. The NxVgMediaWorker.\* log will indicate a fatal error. Replace `redaction.processing.concurrency` with `workflow.processing.concurrency`.

**This should not be an issue for Windows installations *if the InstallVG.ps1 script was used to upgrade*, nor for Linux installations.**

Windows Only: If JAVA\_HOME was not setup before installation, the service start will error out with message like unable to detect java. This should have been taken care of if the InstallVG.ps1 script was used. The service needs to be uninstalled/removed. Re-install using the InstallVG.ps1 script as described in See [Basic Installation / Upgrade \(Windows\)](#)

Windows Only: If the service cannot be stopped, in order to get the service back to a Stopped state, you must

1. Start cmd in admin/elevated mode.
2. Run `sc queryex NxVgMediaWorker`. A PID number will show.
3. Run `taskkill /pid <SERVICE_PID>`

---

## Model Management

---

### Workflow Model Management

---

Voice Grid uses Workbench Workflow Models to perform redaction. These models are stored in the following directory `<NxVgMediaWorker_DATA_ROOT>/models/workflow`. In the workflow directory, there is a named subdirectory for each workflow. Inside that directory are the model files and a manifest file that contains the all-important workflow ID.

---

### Installing new Workflow Models

To install a new model, simply unzip the archive provided to you, and copy the desired models into the `<NxVgMediaWorker_DATA_ROOT>/models/workflow` directory. No additional activity is required to install a model. The next time the application receives a request for a model that has not been cached, the workflow directory will be rescanned and all new models will be installed and cached for immediate use. The newer zip model with extension \*.nxzm.zip does not need to be unzipped and can be placed as is in `<NxVgMediaWorker_DATA_ROOT>/models/workflow` directory

---

### Removing Workflow Models

To remove a workflow model, delete its directory in the `<NxVgMediaWorker_DATA_ROOT>/models/workflow` directory. No further action is required to delete the model. When the application next receives a request for that model, it will detect that it has gone missing, and remove the models that have been removed as well as add those that have been installed.

---

## Call Summarization Model Management

Voice Grid in Call Summarization mode needs models for intents processing and annotation scoring to perform call summarization. These models are given in the call summarization request as request parameters for `textAnalysisModelUri` and `scoringModelUri`. While VoiceGrid can handle multiple models, if you are swapping out the model set being used for a new one, it is recommended to restart all satellites to avoid wasting the memory associated with the models that are no longer being used. Please refer to [Restart Satellites \(restartSatellites\)](#).



If an in-use model is being updated, it is **essential** that all satellites are restarted before processing with that model begins.

---

## Configuration Items

---

### System Level Settings

The following system level configurations are likely candidates for custom modification based on the implementation.



If upgrading from v20.0.0 or earlier to v20.1.0 or later, please see the details about queue name and virtual-host changes near the top of the `README.txt` file, within the "BREAKING CHANGES" subsection, within the "Important Notes for Releases 20.1.0 and Later" section.

---

### General Settings

These are configuration settings common for all implementations.



The "logging.file" property has been deprecated as of Voice Grid 20.0.0 and *should not be used*. Please refer to the [Voice Grid Log Files](#) section for logging configuration details.

### RabbitMQ-Related Settings

Config Key: `spring.rabbitmq.addresses`

Purpose: RabbitMQ instance endpoint. Comma-separated list for a cluster. Supports both DNS host names and IP addresses.

Default Value: `localhost:5672`

Config Key: `spring.rabbitmq.virtual-host`

Purpose: RabbitMQ virtual host used by VG.

Default Value: `vg`



As of Voice Grid v20.1.0, it now defaults to using a virtual host of "vg". Previously this was not set by default.

Config Key: `spring.rabbitmq.username`  
Purpose: Voice Grid's RabbitMQ connection user.  
Default value: `vgclient`

Config Key: `spring.rabbitmq.password`  
Purpose: Voice Grid's RabbitMQ connection password.  
Default value: (see warning below)

The credentials are established during installation and upgrades. They get set for RabbitMQ itself, and for Voice Grid, which is a user of RabbitMQ. Changing these values aside from during an installation or upgrade is discouraged, and if necessary, NiCE support should be contacted for detailed instructions on how to change them to keep RabbitMQ and Voice Grid synchronized.

- **username:** It is highly recommended to leave the default of `vgclient`. Changing requires several changes on RabbitMQ itself in order to set up the required permissions for the new username. NiCE support should be contacted for detailed instructions
- **password:** Changing this is not as complex as it is for the username.



- On Windows, the password is established by the `InstallVG.ps1` script during installation and upgrades. If no password is provided, it will default to `nxVGc13nt` on an installation and will retain the previous password on an upgrade.
- On Linux, the password will be set to the default of `nxVGc13nt`, and will not be modified on an upgrade.
- If the password needs to be changed, it must be changed in Voice Grid's `application.properties`, and using the RabbitMQ management console, and then the NiCE Voice Grid Media Worker service (named `NxVgMediaWorker` if on Linux) must be restarted.

## Other General Settings

Config Key: server.port

Purpose: HTTP port for service status and performance metrics monitoring.

Default Value: 8080

Config Key: workflow.processing.concurrency

Purpose: Min-max (starting and up-to) request worker thread count in accordance with target machine specs (memory and CPU cores). A value lower than the default, such as 4-4, may be more appropriate for Call Summarization implementation mode, and/or if the server has lower specs. A value higher than the default may be more appropriate for Media Analysis mode, particularly on server with higher specs.

Default Value: 12-12

Config Key: media-worker.timeIntervalSecondsBetweenMetricsLogging

Purpose: Time delay in seconds for metrics logging in VG logs. This is to reduce frequently logging thereby reducing the volume of logs and delay their rolling based on size.

Default Value: 120 (900 if running in AWS)

Config Key: voicegrid.requestQueues

Purpose: New property intended to replace the <implementation-specific>.requestQueues properties below.

This is a comma-separated list of request queue names for incoming requests.

If set, it will override the <implementation-specific>.requestQueues value.

Default Value: NOT SET

Config Key: voicegrid.responseQueue

Purpose: New property intended to replace the <implementation-specific>.responseQueue properties below.

This sets the default queue to which Voice Grid will respond if no queue is specified in a request.

If set, it will override the <implementation-specific>.responseQueue value.

Default Value: NOT SET

---

## Workflow Media Analysis Specific Settings

Config Key: media-worker.redact.shutdownWaitTimeSeconds

Purpose: Time limit for graceful service shutdown.

Default Value: 180

Config Key: mediaAnalysis.requestQueues

Purpose: Comma-separated list of request queue names for incoming requests.

Default Value: MediaAnalysisRequest (changed as of VG 20.1)



This property's value may be overridden by voicegrid.requestQueues. See the [Other General Settings](#) section for that property's description.

---

Config Key: mediaAnalysis.responseQueue

Purpose: Public response queue for outgoing responses. Client can specify their private

response queue in the request payload via the clientSystemId field.  
Default Value: MediaAnalysisResponse (changed as of VG 20.1)

---



This property's value may be overridden by voicegrid.responseQueue. See the [Other General Settings](#) section for that property's description.

---

Config Key: media-worker.models.workflow

Purpose: Redaction workflow model repository directory. Can be specified as either absolute or relative path. If relative, it is relative to <NxVgMediaWorker\_DATA\_ROOT>.

Default Value: ./models/workflow

Config Key: media-worker.redact.replacement-string

Purpose: During redaction, the value specified will be put in places of redacted items in the phrases of the output file. If the behavior from VG 2.0.0 or 2.0.1 is desired, the value for this setting should be an empty string.

Default Value: #

---

## CallSummarization Specific Settings



This mode is currently not supported on Linux.

Config Key: implementation

Purpose: This is important for running VG in CallSummarization mode. This new implementation for CallSummarization has been added from VG 3.0.0.

Default Value: MEDIAANALYSIS

Custom Value: CALLSUMMARIZATION - if implementation is CallSummarization

Config Key: callSummarization.satelliteRebootIntervalMin

Purpose: This is for customizing the satellite restart interval using a timer. If this value is non-zero, then the satellites will restart periodically at this interval in minutes.

Restarting satellites is a technique used for ensuring that any memory leaks or progressive Performance problems are alleviated

Default Value: 0 - which means no timer-based restarts

Custom Value: Any value for minutes greater than 0 to restart satellites using a timer.

Similar to above - restart functionality can also be done using the REST api. See [Restart Satellites \(restartSatellites\)](#).

Config Key: callSummarization.clientFailuresBeforeRestart

Purpose: Threshold value of total client failures after which satellites will be automatically restarted.  
A negative value disables this feature.

Default Value: -1

Config Key: callSummarization.restartsBeforeDisabled

Purpose: Threshold value of total satellite restarts after which a satellite needs to be disabled.  
This setting is to limit the times a satellite is restarted after failing before being disabled.

---



These restarts may be triggered by `clientFailuresBeforeRestart` OR if the satellite fails immediately when started, regardless of the `clientFailuresBeforeRestart` setting.  
A negative value disables this feature.

Default Value: 3

Config Key: `callSummarization.topIntents`

Purpose: The number of Intents to be captured and returned as `CustomerIntents`. A value of 1 captures a single Primary Intent. A larger value returns the Primary Intent and (n-1) additional Top Intents.

Default Value: 3 - Primary Intent and 2 additional Top Intents

Config Key: `callSummarization.requestQueues`

Purpose: Comma-separated list of request queue names for incoming requests.

Default Value: `CallSummarizationRequest`



This property's value may be overridden by `voicegrid.requestQueues`. See the [Other General Settings](#) section for that property's description.

Config Key: `callSummarization.responseQueue`

Purpose: Public response queue for outgoing responses. Client can specify their private response queue in the request payload via the `clientSystemId` field.

Default Value: `CallSummarizationResponse`



This property's value may be overridden by `voicegrid.responseQueue`. See the [Other General Settings](#) section for that property's description.

Config Key: `media-worker.nlpSatelliteBox.host`

Purpose: The host for Nlp Satellite which will be spun up during the processing.

Default Value: `localhost`

Config Key: `media-worker.nlpSatelliteBox.port`

Purpose: This value will be the port for first instance of Nlp satellite. It will then use the next consecutive port up to the concurrency requested for the next instances of Nlp satellites.

Default Value: 51200

Config Key: `media-worker.textAnalysis.host`

Purpose: The host for Text Analysis Satellite which will be spun up during the processing.

Default Value: `localhost`

Config Key: `media-worker.textAnalysis.port`

Purpose: This value will be the port for first instance of Text Analysis satellite. It will then use the next consecutive port up to the concurrency requested for the next instances of Text Analysis satellites.

Default Value: 51300

Config Key: `callSummarization.textAnalysisSatelliteDir`

Purpose: To be used only in case you want to use a different Text Analysis Satellite than the one included with Voice Grid

Default Value: `<install-directory>/NxVgMediaWorker/satellites/IntentAnalysis`

Custom Value: `<Satellite_Install_Location>/IntentAnalysis`

`<Satellite_Install_Location>` would be the new location where Text Analysis satellite will be installed.

Config Key: `callSummarization.nlpSatelliteDir`

Purpose: To be used only in case you want to install the Nlp Satellite outside of the Voice Grid install location. Default satellite locations will work for most installations.

Default Value: `<install-directory>/NxVgMediaWorker/satellites/nlpbox/nlpbox-satellite-<version>`

Custom Value: `<Satellite_Install_Location>/nlpbox/nlpbox-satellite-<version>`

`<Satellite_Install_Location>` would be the new location where Nlp satellite will be installed.

Config Key: `media-worker.satelliteTimeoutInMsec`

Purpose: Used to increase the timeout in milliseconds. If more satellites are being spun up essentially simultaneously, the longer each satellite takes to spin up. This can be increased to avoid timeout that can occur during satellite startup.

Default Value: 600000

Config Key: `callSummarization.nlpSatelliteDir.keFile`

Purpose: To replace the default model (ke file) for the nlp satellite

Default Value: `ke\summarization_domain-22.4.1.May.01.2025.ke`

Config Key: `callSummarization.textAnalysisSatellite.tmpDir`

Purpose: To replace the temp directory for Text Analysis Satellite process.

Default Value: Location defined in %TMP% setting on windows

Config Key: `callSummarization.resultfile.override`

Purpose: To override the default behavior of overwriting results file when given in resultUri.

Default Value: true - Overwrite Results file is default behavior to enable reprocessing same requests again



The former `media-worker.shutdownWaitTimeSeconds` property has been deprecated as of Voice Grid 21.0.1 and replaced with `media-worker.shutdownWaitTimeSecondsAddedToConcurrencyMinutes` described below.

Config Key: `media-worker.shutdownWaitTimeSecondsAddedToConcurrencyMinutes`

Purpose: To extend the calculated shutdown timeout, if necessary, in order to allow for a clean and graceful shutdown of Voice Grid and the satellites, allowing all in-flight requests to complete and satellite processes to be stopped cleanly. (If set to a negative value, it can also shorten the calculated time, but that is not recommended in most instances.)

Specifically the maximum shutdown wait time is calculated as follows:

$\text{maxSecondsToWait} = (\text{concurrency} * 60) + \text{shutdownWaitTimeSecondsAddedToConcurrencyMinutes}$

For example, if concurrency is set to 4-4, and this property is using its default value of 120 seconds, then:

$(4 * 60) + 120 = 360$  seconds (or 6 minutes)

It should be noted that the Windows Services GUI will only wait for about 2 minutes before it reports that service did not stop in a timely manner. That does NOT indicate that Voice Grid needs to be forcibly stopped using Task Manager or anything else. It is very important to let Voice Grid stop on its own, especially in CallSummarization mode.

In most cases, the default value should be fine, but extending it could be necessary for some installations, depending customer data and server processing power. If it is taking longer than the calculated time to shut down, Voice Grid will forcibly terminate processes in order to guard against a hung condition, but unnecessarily forced shutdowns should be avoided because they can leave orphaned satellite processes, which occupy gigabytes of RAM and gigabytes of disk space, though no data will be lost and in-flight requests will be re-processed once Voice Grid is restarted.

Default Value: 120 seconds

---

## ElevateAI Specific Settings

Config Key: implementation

Purpose: This is important for running VG in ElevateAI mode.

This new implementation for ElevateAI has been added from VG 4.0.0

Default Value: MEDIAANALYSIS

Custom Value: ELEVATEAI - if implementation is ElevateAI

Config Key: workflow.processing.mode

Purpose: processing mode can be realtime or offline.

Default set to offline (post-call batch processing of media files)

Default Value: offline

Config Key: offline.mediaUriReplacementPattern

Purpose: On-Prem/AWS interoperability

Default Value:

Config Key: offline.copyMediaToLocalDirectory

Purpose: AWS performance issue work-around

If not set, media will not be copied to a local directory.

If set to a relative path, that path is relative to <NxVgMediaWorker\_DATA\_ROOT>

If set to an absolute path, that path must not be within <install-directory>

Default Value:

Config Key: offline.detectMirroredStereo

Purpose: mirror detection logic related config

Default Value: false

Config Key: offline.detectMirroredStereoDurationSeconds

Purpose: mirror detection logic related config

Default Value:

Config Key: offline.requestQueues

Purpose: Request/response queue names override.

Default Value:



This property's value may be overridden by `voicegrid.requestQueues`. See the [Other General Settings](#) section for that property's description.

Config Key: `offline.responseQueue`

Purpose:

Default Value:



This property's value may be overridden by `voicegrid.responseQueue`. See the [Other General Settings](#) section for that property's description.

Config Key: `offline.deadLetterQueue`

Purpose: Request/response queue names override.

Default Value:

Config Key: `realtime.requestQueue`

Purpose: Request/response queue names override.

Default Value:

Config Key: `workflow.processing.modelCachingLimit`

Purpose:

Default Value:

Config Key: `offline.skipInChannelPayload`

Purpose:

Default Value:

Config Key: `offline.shutdownWaitTimeSeconds`

Purpose: Graceful shutdown threshold

Default Value: 3600 (60 minutes)

Config Key: `realtime.defaultWorkflow`

Purpose:

Default Value:

Config Key: `workflow.processing.continueOnDiarizationFailure`

Purpose: Sometimes diarization fails for single channel media files. Workflows can override the behavior and force all media files to be processed even if the diarization process fails, by setting this value to true

Default Value: true

Config Key: `workflow.processing.modelRepo`

Purpose: Workflows model repo location. It should generally be customized to a centralized network location

Default Value: <Installation Directory>/model/workflows

Config Key: media-worker.resultfile.override

Purpose: To override the default behavior of overwriting results file when given in resultUri.

Default Value: true - Overwrite Results file is default behavior to enable reprocessing same requests again, set to false to avoid overwriting the resultfile.



Specify model file paths using forward slash (/). Misconfiguration will prevent the service from starting up. (Check service logs for errors.)

---

## Transcription Specific Settings

Config Key: implementation

Purpose: This is important for running VG in Transcription mode.

This new implementation for Transcription has been added from VG 21.0.0

Default Value: MEDIAANALYSIS

Custom Value: TRANSCRIPTION - if implementation is Transcription

Config Key: transcription.requestQueues

Purpose: Comma-separated list of request queue names for incoming requests.

Default Value: TranscriptionRequest

Config Key: transcription.responseQueue

Purpose: Public response queue for outgoing responses. Client can specify their private response queue in the request payload via the clientSystemId field.

Default Value: TranscriptionResponse

Config Key: media-worker.transcription.host

Purpose: The host for the T11 Transcription satellite used for processing audio files.

Default Value: localhost

Config Key: media-worker.transcription.port

Purpose: The port for the T11 Transcription satellite.

Default Value: 50050

Config Key: media-worker.transcription.alterationsConfigFile

Purpose: Provides default phrase boosting and transcription result replacement strings to be used if none are specified with the request from the client (DEF).

NOTE: If the file does not exist, then there are no defaults. The file content is not intended to be modified. Contact NiCE to get a customer/site-specific file that is properly formatted.

Default Value: TranscriptionAlterations/config.json

This default gets populated ONLY IF a corresponding file is in S3 for the site.

Config key: media-worker.transcription.defaultTranscriptionLanguage

Purpose: Sets the default transcription language for when transcription requests are sent from the client (DEF) with no languages specified. This is the ISO 639-1 two-letter code, optionally with an ISO 3166-1 country/region code appended.

NOTE: The T11 Transcription Satellite supports most languages, but very few ISO 3166-1 country/region codes. This is the full set supported as of v11.3.1:

af, am, ar, ar-acm, ar-afb, ar-ajp, ar-apc, ar-arb, ar-arq, ar-arz, as, az, ba, be, bg,

bn, bo, br, bs, ca, cs, cy, da, de, el, en, en-AU, en-GB, en-US, es, es-419, es-AR, es-ES, et, eu, fa, fa-AF, fa-IR, fi, fo, fr, fr-CA, fr-EU, gl, gu, ha, haw, he, hi, hr, ht, hu, hy, id, is, it, ja, jw, ka, kk, km, kn, ko, ku-ckb, la, lb, ln, lo, lt, lv, mg, mi, mk, ml, mn, mr, ms, mt, my, ne, nl, nl-BE, nl-NL, nn, no, oc, pa, pl, ps, pt, pt-BR, ro, ru, sa, sd, si, sk, sl, sn, so, sq, sr, su, sv, sw, ta, te, tg, th, tk, tl, tr, tt, uk, ur, uz, vi, yi, yo, yue, zh, zh-cmn, zh-yue

Default Value: en

---

## Concurrency Control

Voice Grid utilizes a pool of worker threads (for each type of operation, e.g. Redaction) to process batches of requests dispatched from the message broker. The pool size is configured via config key **workflow.processing.concurrency**. By default, its value is set to "12-12" which roughly equates 12 workers. It is recommended that the number of workers should not exceed the number of logical CPU cores and each worker gets 1GB of main memory on the host system.

When implemented in CallSummarization mode, the number of satellite pairs spun up is configured via config key **workflow.processing.concurrency**. Most of our Call Summarization testing has been done with settings of "4-4", which equates to 4 workers and 4 instances each of Nlp and Text Analysis satellites, so that is recommended at this point. The default setting of "12-12" may be fine on powerful servers, but has not had extensive testing. Note that even though the minimum number of satellite pairs is marked in the concurrency operation, when starting satellite pairs, a maximum of two pairs will be starting at any given time, in order to ramp up the startup time of the various satellites.



Initially, we recommend reserving 2 cores for OS and VG overhead, so not setting the concurrency higher than the number of physical cores – 2 for workflow processing. For Call Summarization, since there are multiple processes involved in the various satellites that do call summarization, we recommend starting with 4 for concurrency, and not exceeding  $(\text{number of physical cores} - 2) / 3$ . For example, for 32 cores, reserve 2 for OS & VG overhead, leaving 30. Divide by 3 for a concurrency of 10-10.

- Due to the light-weighted nature of the idle threads and to simplify administration, set the concurrency min and max to equal count. Service will warn against (but not forbid) settings in exceed of apparent system capacities; but it is highly recommended that a client deployment reconfigure this item to a value not exceeding the physical CPU core count.

---

## GPU Use

By default, Voice Grid will try to use GPUs for phrase recognition if any are present. If for any reason GPU usage needs be disabled (for example, if a non-certified GPU is installed and causing issues), add the following configuration item:

**nexidia.workbench.disableGpu=true**

The status of GPU usage is logged at service startup.

---

## Multiple Voice Grid Instances

Voice Grid exhibits almost linear throughput scaling characteristics when installed on a cluster of equally spec'd servers. Due to the light-weighted processing involved in message queuing and dispatching, and the relatively small sizes of service request/response payloads, the centralized message broker is unlikely to become a bottleneck unless a great number of service and client instances are connected to it at the same time (up to 48 concurrent connections have been tested and no issues were found.)

There is no added complexities in a clustered Voice Grid deployment. Each instance is installed and operates independently. The only caution is to make sure their deployed directory structure and file content are identical.

---

## Monitoring and Health Check

System message processing performance metrics can be observed on the RabbitMQ Management console. Individual Voice Grid instances can be monitored using standard Windows services utilities, examining the application logs, and using HTTP endpoints to query the service status and performance information.

---

## Voice Grid Log Files

The logs directory is automatically generated after installation. There are separate files for service PID, the Installation Log (NxVgMediaWorker\_PR\*, generated by the Apache procrun service wrapper) and the Service Log (NxVgMediaWorker.\*). The PR logs are only changed when the Voice Grid service is installed or removed; whereas the service log grows as request traffic comes in.

```
[NxVgMediaWorker_DATA_ROOT>]
/logs
 NxVgMediaWorker.<date stamp>.log
 NxVgMediaWorker_PR.<date stamp>.log
 pid.txt
```

Voice Grid service log files roll when their uncompressed size reach 25 MiB, and up to 10 old ones are retained as zip files. This is the default rolling policy set in logback-spring.xml file found in the installation directory at same level as the application.properties. The rolling policy can be customized based on the needs of the client installation. Below are the settings relating to log retention that can be customized. Default values are shown:

```
<maxIndex>10</maxIndex>
<MaxFileSize>25MB</MaxFileSize>
```

The verbosity of logging information is also controlled in the logback-spring.xml file. Below is the default setting:

```
<logger name="com.nexidia" level="INFO"/>
```

For troubleshooting purposes only, this can be changed to "DEBUG", which will enable debug logging in *all* NiCE code. (All NiCE code in Voice Grid is packaged under com.nexidia.) With this set to "DEBUG", both additional service request processing steps and workbench workflow api internal diagnostic info will be logged.

Alternatively, to specify a particular logging level to set for any package or class, additional, similar lines can be added immediately below that line.



The updates to logback-spring.xml can be done while the VoiceGrid Service is running. The changes take effect immediately and there is no need for the Service to be stopped or restarted.

---

## Additional CallSummarization Logs

There are several different locations where CallSummarization logs will show up and need to be examined to do troubleshooting.

Location	Comments
<NxVgMediaWorker_DATA_ROOT>/logs	General VoiceGrid logs
<NxVgMediaWorker_DATA_ROOT>/nlpbox/logs	General nlp satellite logs
<NxVgMediaWorker_DATA_ROOT>/nlpbox/logs/ nlp<port#>Errors.log	Stderr from a specific nlp satellite
<NxVgMediaWorker_DATA_ROOT>/nlpbox/logs/ nlp<port#>Logs.log	stdout from a specific nlp satellite
<NxVgMediaWorker_DATA_ROOT>/textAnalysis/logs	General textAnalysis satellite logs
<NxVgMediaWorker_DATA_ROOT>/textAnalysis/logs/ logs<port#>/textAnalysis<port#>Errors.log	stderr from a specific text analysis satellite
<NxVgMediaWorker_DATA_ROOT>/textAnalysis/logs/ logs<port#>/textAnalysis<port#>Logs.log	stdout from a specific text analysis satellite

---

## HTTP Endpoints (REST APIs) for Monitoring and Checking Health

Voice grid has several REST APIs that can be useful for basic querying of service status and performance information. Each are documented below, and their implementation and responses may vary depending on which mode that Voice Grid is in (MediaAnalysis, CallSummarization or ElevateAI).



The examples show the use of the application, `curl` as a means of calling those APIs from a command prompt, though for all the GET APIs, any web browser can be used; it just can't be used for the Restart Satellites API, which requires a POST operation.

- Referring to the examples below, to query a remote server, simply replace `localhost` with the IP address of the server you wish to query.
- The default port is `8080`, but if you have configured a different port, substitute that port instead of `8080`.

---

## Get Health Information (actuator/health)

### Request health of the Voice Grid system

```
$ curl 'http://localhost:8080/actuator/health' -i -X GET
```

### Example Health Responses

Where the throughput (xRT) is provided, it consists of following measures:



- Observed Node Throughput, defined as total processed media length divided by server uptime
- Implied Single Worker Throughput, defined as total media length divided by total processing time.

### Example Health Response (MediaAnalysis Mode)

```
HTTP/1.1 200 OK
Content-Type: application/vnd.spring-boot.actuator.v3+json

{
 "status" : "UP",
 "components" : {
 "diskSpace" : {
 "status" : "UP",
 "details" : {
 "total" : 858345435136,
 "free" : 403375386624,
 "threshold" : 10485760,
 "exists" : true
 }
 },
 "ping" : {
 "status" : "UP"
 }
 }
}
```

```

 },
 "rabbit" : {
 "status" : "UP",
 "components" : {
 "mediaAnalysisResponseSender" : {
 "status" : "UP",
 "details" : {
 "version" : "3.13.0"
 }
 },
 "testMessenger" : {
 "status" : "UP",
 "details" : {
 "version" : "3.13.0"
 }
 }
 }
 },
 "workflowRequestExecutor" : {
 "status" : "UP",
 "details" : {
 "metrics": "Uptime(sec): 1423, Requests Processed: 100, Failed: 0, In-
flight: 0, Throughput(xRT): 103.00/14.05"
 }
 }
 }
}

```

### Example Health Response (CallSummarization Mode)

```

HTTP/1.1 200 OK
Content-Type: application/vnd.spring-boot.actuator.v3+json

{
 "status" : "UP",
 "components" : {
 "callSummarizationRequestExecutor" : {
 "status" : "UP",
 "details" : {
 "metrics": "Uptime(sec): 3778, Requests Processed: 5, Failed: 0, In-
flight: 0, Throughput(xRT): 0.00/1.00"
 }
 },
 "diskSpace" : {
 "status" : "UP",
 "details" : {
 "total" : 858345435136,
 "free" : 403375378432,

```

```

 "threshold" : 10485760,
 "exists" : true
 },
 "ping" : {
 "status" : "UP"
 },
 "rabbit" : {
 "status" : "UP",
 "components" : {
 "callSummarizationResponseSender" : {
 "status" : "UP",
 "details" : {
 "version" : "3.13.0"
 }
 },
 "testMessenger" : {
 "status" : "UP",
 "details" : {
 "version" : "3.13.0"
 }
 }
 }
 }
}

```

---

In CallSummarization mode, if any satellite restarts or failures have occurred, additional metrics will be provided, as shown below:

---

```

"metrics": "Uptime(sec): 3778, Requests Processed: 5, Failed: 0, In-flight: 0, Throughput(xRT): 0.00/1.00,
Satellites restarted 2, Satellite Failure Detected 0"

```

### Example Health Response (ElevateAI Mode)

```

HTTP/1.1 200 OK
Content-Type: application/vnd.spring-boot.actuator.v3+json

{
 "status": "UP",
 "components": {
 "diskSpace": {
 "status": "UP",
 "details": {
 "total": 858345435136,
 "free": 394797744128,

```

```

 "threshold": 10485760,
 "exists": true
 }
},
"elevateWorkflowRepo": {
 "status": "UP",
 "details": {
 "workflows": {
 "found": {
 "65b064cc-3972-4673-8f14-571ba95f6e8d": "file://vanxproserv18/
ELEVATE_DATA/workflows/v1.2/IntlEnglish/ElevateAI_EIA_EIA_en_WB10.8_vv2.nxzm.
zip",
 "a9880a7f-ce2e-4813-ab98-c43e97ef70a3": "file://vanxproserv18/
ELEVATE_DATA/workflows/v1.2/IntlEnglish/ElevateAI_INTL_Large_EIA_en_WB10.8_v1.
2.nxzm.zip",
 "686a78ba-5c4b-4d82-b16a-396ac9bc2a55": "file://vanxproserv18/
ELEVATE_DATA/workflows/v1.2/NASpanishLarge/ElevateAI_EIA_Large%20Spanish_es-
419_WB10.8_v1.nxzm.zip",
 "d3eaf4d1-bc07-4b5f-808e-e4d75503159b": "file://vanxproserv18/
ELEVATE_DATA/workflows/v1.2/NAE/ElevateAI_EIA_NAE_en-US_WB10.8_v2.nxzm.zip",
 "f306c1a1-0bce-4c09-b6fc-43499116fa34": "file://vanxproserv18/
ELEVATE_DATA/workflows/v1.2/NAE/ElevateAI%20_ElevateAI_Large_EIA_en-US_WB10.8_
v1.2.nxzm.zip",
 "03e3d480-6840-4f30-b520-0c3f78d712a7": "file://vanxproserv18/
ELEVATE_DATA/workflows/v1.2/LIDBeta/Elevate_NV_EIA__WB10.8_v1.nxzm.zip"
 },
 "repoRoot": "//vanxproserv18/ELEVATE_DATA/workflows/v1.2",
 "loaded": [],
 "pattern": "\\w.*\\. (nxzm\\.zip|flow) "
 }
 }
},
"ping": {
 "status": "UP"
},
"rabbit": {
 "status": "UP",
 "components": {
 "mediaAnalysisResponseSender": {
 "status": "UP",
 "details": {
 "version": "3.13.0"
 }
 },
 "testMessenger": {
 "status": "UP",
 "details": {
 "version": "3.13.0"
 }
 }
 }
}

```

```

 }
 }
},
"workflowMediaAnalyzer": {
 "status": "UP",
 "details": {
 "metrics": {
 "Uptime(sec)": 71,
 "TextProc Rate(docs/min)": {
 "Last5min": 0,
 "Last10min": 0,
 "Last1min": 0,
 "Since up": 0
 },
 "Audio Throughput(xRT)": {
 "Overall": 0,
 "Unfolded": 0
 },
 "In-flight": 0,
 "Failed": 0,
 "Processed": 0
 }
 }
}
}
}
}

```

### Example Health Response (Transcription Mode)

```

HTTP/1.1 200 OK
Content-Type: application/vnd.spring-boot.actuator.v3+json

{
 "status": "UP",
 "components": {
 "diskSpace": {
 "status": "UP",
 "details": {
 "total": 214668652544,
 "free": 64560480256,
 "threshold": 10485760,
 "path": "/opt/Nexidia/VoiceGrid/NxVgMediaWorker/.",
 "exists": true
 }
 },
 "ping": {
 "status": "UP"
 }
 },
}

```

```

"rabbit": {
 "status": "UP",
 "components": {
 "testMessenger": {
 "status": "UP",
 "details": {
 "version": "3.13.7"
 }
 },
 "transcriptionResponseSender": {
 "status": "UP",
 "details": {
 "version": "3.13.7"
 }
 }
 }
},
"ssl": {
 "status": "UP",
 "details": {
 "validChains": [],
 "invalidChains": []
 }
},
"transcriptionRequestExecutor": {
 "status": "UP",
 "details": {
 "transcriptionSatelliteStatus": "SERVING",
 "metrics": {
 "Failed": 0,
 "In-flight": 0,
 "Processed": 3,
 "ThroughputDetails": {
 "Media": {
 "DurationSeconds": 387,
 "PerSecond": {
 "Last01min": "0.00",
 "Last05min": "0.00",
 "Last15min": "0.00",
 "Mean": "0.00"
 }
 },
 "Processed": 3,
 "xRT": {
 "Last01min": "0.00",
 "Last05min": "0.00",
 "Last15min": "0.01",
 "Mean": 0.05
 }
 }
 }
 }
}

```

```

 },
 "ThroughputSummary": {
 "MediaProcRate(xRT)": {
 "Overall": "0.07",
 "Per-Request": "9.33"
 }
 },
 "Uptime(sec)": 8096,
 "ValuesReset": "No"
 }
}
}
}
}
}

```

---

## Get Resource Usage (actuator/resourceUsage)

The resourceUsage actuator endpoint can be used to query CPU and memory usage information.

### Request Memory Usage Info

#### Curl Request

```
$ curl 'http://localhost:8080/actuator/resourceUsage' -i -X GET
```

or

```
curl -s http://localhost:8080/actuator/resourceUsage
```

#### HTTP Request

```
GET /actuator/resourceUsage HTTP/1.1
Host: localhost:8080
```

### Memory Usage Info Response

#### Example Memory Usage Info Response Header

```
Content-Type: application/vnd.spring-boot.actuator.v3+json
Transfer-Encoding: chunked
Date: Sat, 12 Jul 2025 01:00:06 GMT
```

## Example Memory Usage Info Response Body

```
{
 "Process": {
 "CPU": {
 "Load(%)": "3"
 },
 "Memory": {
 "JavaHeap(MiB)": {
 "Current": "112.00",
 "Peak": "112.00"
 },
 "Overall(GiB)": {
 "CommitCharge": {
 "Current": "9.12",
 "Peak": "9.71"
 },
 "WorkingSet": {
 "Current": "5.59",
 "Peak": "6.43"
 }
 }
 }
 },
 "System": {
 "CPU": {
 "Load(%)": "12"
 },
 "Memory": {
 "CommitPercentages": {
 "OfLimit": "62.35",
 "OfPhysicalRAM": "66.25"
 },
 "Values(GiB)": {
 "CommitLimit": "68.00",
 "Committed": "42.40",
 "PhysicalRAM": "64.00"
 }
 }
 }
}
```

---

## Get Inventory

### MediaAnalysis Mode Inventory (inventory)



## Request the inventory of installed models (MediaAnalysis Mode)

```
$ curl 'http://localhost:8080/inventory' -i -X GET
```

## Example Inventory Response (MediaAnalysis Mode)

```
HTTP/1.1 200
Content-Type: application/json
Transfer-Encoding: chunked

{
 "workflows": {
 "9c1297a5-6880-412a-8b71-c4cf915707bc": {
 "description": "asr.naenglish.mattersight.WB10.2",
 "path": "ASR workflow for NAEnglish"
 },
 "a315c30c-20e2-4a8f-a7ca-48ffe6d102ab": {
 "description": "asr.redaction.naenglish.mattersight.default.WB10.2",
 "path": "ASR with Redaction workflow for NAEnglish to match as close as possible Mattersight default v1"
 },
 "fd5c1b01-22c6-4b30-a665-5ca762fc943f": {
 "description": "asr.redaction.naenglish.mattersight.credit_card_default.WB10.2",
 "path": "ASR with Redaction workflow for NAEnglish to match as close as possible Mattersight Credit Card Default V1"
 },
 "282dd302-7e29-40c6-9358-b453bff340ad": {
 "description": "asr.redaction.naenglish.mattersight.credit_card_default.WB10.2",
 "path": "ASR with Redaction workflow for NAEnglish to match as close as possible Mattersight Credit Card Default V2"
 }
 }
}
```

## ElevateAI Mode Inventory (api/workflows)

### Request the inventory of installed models (ElevateAI Mode)

```
$ curl 'http://localhost:8080/api/workflows' -i -X GET
```

### Example Inventory Response (ElevateAI Mode)

```
HTTP/1.1 200 OK
```

Content-Type: application/json

```
[
 {
 "guid": "65b064cc-3972-4673-8f14-571ba95f6e8d",
 "language": "en",
 "description": "En with SE/CSAT/VE models, Large transformer",
 "path": "WORKFLOW_MODEL_65b064cc-3972-4673-8f14-571ba95f6e8d/ElevateAI_en_
large.xml",
 "modelVertical": "default",
 "modelTranscriptionMode": "high_accuracy"
 },
 {
 "guid": "a9880a7f-ce2e-4813-ab98-c43e97ef70a3",
 "language": "en",
 "description": "large transformer intl english",
 "path": "WORKFLOW_MODEL_a9880a7f-ce2e-4813-ab98-c43e97ef70a3/ElevateAI_
ElevateAI_Large_EIA_en_intlEnglish_WB10.8_v1.2..xml",
 "modelVertical": "default",
 "modelTranscriptionMode": "high_accuracy"
 },
 {
 "guid": "03e3d480-6840-4f30-b520-0c3f78d712a7",
 "language": "En_ES419",
 "description": "LID for elevate Eng/ES",
 "path": "WORKFLOW_MODEL_03e3d480-6840-4f30-b520-0c3f78d712a7/en_es_lid.xml"
 },
 {
 "guid": "f306c1a1-0bce-4c09-b6fc-43499116fa34",
 "language": "en-US",
 "description": "Large transformer ElevateAI",
 "path": "WORKFLOW_MODEL_f306c1a1-0bce-4c09-b6fc-43499116fa34/ElevateAI_NAE_
LargeTransformer_CX1Training.xml",
 "modelVertical": "default",
 "modelTranscriptionMode": "high_accuracy"
 },
 {
 "guid": "d3eaf4d1-bc07-4b5f-808e-e4d75503159b",
 "language": "en-US",
 "description": "en-us large 10.8.3 with enl adding SE ",
 "path": "WORKFLOW_MODEL_d3eaf4d1-bc07-4b5f-808e-e4d75503159b/ElevateAI_NAE_
Large_v2.xml",
 "modelVertical": "default",
 "modelTranscriptionMode": "high_accuracy"
 },
 {
 "guid": "686a78ba-5c4b-4d82-b16a-396ac9bc2a55",
 "language": "es-419",
 "description": "es-419",

```

```

 "path": "WORKFLOW_MODEL_686a78ba-5c4b-4d82-b16a-396ac9bc2a55/ElevateAI_
Large_NASpanish.xml",
 "modelVertical": "default",
 "modelTranscriptionMode": "high_accuracy"
 }
]

```

## Restart Satellites (restartSatellites)



Applies *only* to CallSummarization mode. This API cannot be used via a web browser.

Restart **all** the Call Summarization Satellites. A good use case is when models are changed after processing for some time, all satellites can be restarted before resuming to process with newer models, avoiding wasting the memory associated with models that are no longer being used.

```
$ curl 'http://localhost:8080/restartSatellites' -i -X POST
```

Similar functionality in case needed at set intervals can be achieved using the `callSummarization.satelliteRebootIntervalMin` property. See [CallSummarization Specific Settings](#) for more information in this regard.

There is a convenience powershell script `restartSatellites.ps1` installed with VoiceGrid that makes the RestartSatellites call.

```
restartSatellites [-server <server ip or name>] [-port <server port>]
```

By default, it will send the REST command to localhost:8080.

This command can be used in concert with Windows Task Scheduler to set the CallSummarization Grid to restart all the satellites each night at a particular time, or can be used interactively if lagging performance is detected.

## Voice Grid in Call Summarization Implementation Mode (Windows Only)

### Installation

Same instructions for installation as a regular Voice Grid - detailed in [Basic Installation / Upgrade \(Windows\)](#), but pass "-VGImpl CallSummarization" the command-line to the InstallVG.ps1 PowerShell script.

Even if you installed VoiceGrid on a drive other than C, by default, the python executables actually run in the Windows %TMP% directory (typically C:\users\<service-user-name>\AppData\Local\Temp). If your C drive does not have a lot of extra space, then you may need to change the below line in application.properties to point to a different hard drive.



If set to a relative path, that path is relative to <NxVgMediaWorker\_DATA\_ROOT>, so if Voice Grid was installed with <data-directory> on hard drive other than C, you can set this to a relative path, for example, to `./temp` and it will use <NxVgMediaWorker\_DATA\_ROOT>/temp as the location.

---

`#callSummarization.textAnalysisSatellite.tmpDir=`

---

## Running the Call Summarization Grid

Start the Voice Grid Media Worker service. Call Summarization Requests can now be sent to Voice Grid.

The message needs to be a JSON string in the following format:

```
{
 "clientId": null,
 "requestTimestamp": 1537132507296,
 "requestId": "bb13ba8f-5ce4-484d-8ca2-a8af33a60206",
 "transcriptUri": "//nas3/Engineering/TestData/transcripts/Transcript748667.CSV",
 "textAnalysisModelUri": "//nas3/Engineering/TestData/model/DefaultIntents.ipm",
 "scoringModelUri": "//nas3/Engineering/TestData/model/actionScoringModel.asm",
 "resultUri": "D:/output/result.zip",
 "sendSatelliteLogs": false
}
```

The `clientId`, `transcriptUri`, `textAnalysisModelUri`, and `scoringModelUri` all need to be replaced before sending the request. `clientId` is needed only when a private response queue is preferred other than the default `CallSummarizationResponse` queue. `resultUri` is optional when a local copy of `callSummarizationResponse` is required to be written to path `Uri` and file name defined in zip format. `sendSatelliteLogs` is also optional and default is false. When sent in request as true, will get relevant satellite logs in response. Copies of both the Text Analysis Model and Scoring Model can be found at `\\nas3\Engineering\IntentAnalysis-TestData\models`.

The default request and response queue names in the RabbitMQ instance are `CallSummarizationRequest` and `CallSummarizationResponse`.

---

## Troubleshooting

If the `CallSummarizationGrid` has stopped or crashes, then multiple instances of the satellites may be left running. Before restarting the `CallSummarizationGrid`, kill these by running the `bin/killCallSummarizationSatellites.ps1` script.

During the request processing, satellites run locations for logs are created and can fail if the file/directory permissions are not properly set. The VG logs will show such reason like "Unable to set execute permissions" or "Unable to create directory" issues. The required permissions need to be set and VG should be restarted to resolve the issue.

For each active CallSummarization, there will be an instance of both satellites. Thus, the server may run out of memory if too many simultaneous requests are being handled. This may be throttled using the concurrency setting property in application.properties like shown below:

```
| workflow.processing.concurrency=12-12
```

where the first 12 is starting **minimum** threads, and second 12 is **maximum** threads. (Refer to the [Concurrency Control](#) and [General Settings](#) sections for additional information on this setting.)



This number of threads also limits the number of satellites that will be started. Testing using a different number of starting and maximum threads is not yet complete, so it is recommended to make the two values the same.

If the demand is such that more than 2 satellite pairs should be spun up simultaneously, VG will ramp up this startup, so the first two will attempt to start at first, but the third pair will not start until the first one is complete in its spin up.

The maximum time allowed for this spin-up is set by the property

```
| #media-worker.satelliteTimeoutInMsec=600000
```

We have run into some instances where a port is already in use causing a satellite to not start up and this is visible in the logs.

You can check to see what is using a port using

```
| netstat -ano|findstr <port #>
```



The ports being used are 51200–5120x and 51300–5130x ( where x is how many simultaneous workers are running )

---

## Common Exception Protocol (CEP) Server Configuration

NiCE's CEP Server product runs on Windows only, however, Voice Grid, whether installed on Windows or Linux, can be enabled to connect to NiCE's CEP Server product. This allows Voice Grid to send periodic heartbeats that the CEP server can monitor, as well as sending any error (or in some cases warning) log messages for the CEP server to store. All messages sent to the CEP server are encrypted during transport.

By default, CEP is disabled in Voice Grid, but can be enabled using either the default self-signed certificate that the CEP Server creates for clients when it is installed, or a custom (customer-provided) certificate.



To use a custom certificate, that certificate will also have to be installed on the CEP server. This document only covers configuring CEP within Voice Grid and installing certificates into Voice Grid. Please refer to the CEP Server documentation for its certificate installation instructions.

---

## A Little About Certificates

A certificate is used not only to encrypt the data using its public key (readable only by a receiver that has the matching private key), but also as a means of authenticating that the server is in fact the server that the client is expecting. To provide that, the certificate is signed, and the client validates that signature; however all signatures are not created equal.

Signature chains are used to validate authenticity. For example, the certificate used on a server may have been issued by an organization that is not a trusted root authority, but that organization has been authenticated by another organization, and so on up the chain to a trusted root authority.

- **Self-Signed Certificates:** The lowest-level of authentication is a self-signed certificate, which is not issued by any authority. As its name implies, anyone can create a self-signed certificate, and thus it really provides no guarantee of the server's identity; however it does allow data to be exchanged in an encrypted format. If an organization is extremely confident of its network security and the inability for any unauthorized devices to appear on its network, a self-signed certificate may be sufficient, but some organizations will not be satisfied with a self-signed certificate. The CEP server provides a self-signed certificate to copy to clients, such as Voice grid, for them to use without an organization *needing* to generate their own certificate. An organization could generate their own self-signed certificate and use that, but given that it adds no additional level of security, it's fairly pointless vs. just using the self-signed certificate generated by the CEP server.
- **Internally-Signed Certificates:** The next level of authentication is similar to a self-signed certificate, but is signed within the organization, and keyed specifically to the server on which it is to be installed. That certificate is used for encryption. To authenticate the encryption certificate, that organization then typically installs another certificate of its own – the one it uses to sign its other certificates — on all of its internal computers as a trusted root certificate, even though it's not truly authenticatable through a chain back to a trusted root authority. This type of internally-signed authentication certificate is popular on servers that are only internally-accessible because the outside world does not need to authenticate them up to a trusted root authority, and internally-signed certificates are free, whereas getting a certificate signed by an external trusted authority has a cost associated with it. In this case, the internally-signed certificates for the individual servers authenticate up the chain (usually just one level) to the other certificate that the organization has placed on its computers as a trusted root certificate. Within the organization, client apps are satisfied with authentication. The level of security, however, depends upon how securely that other "trusted" certificate is managed by the organization. To acquire the server-specific signed certificate, some organizations may have an IT Security department that creates the certificates upon request, while others may have the requester create certificate and keypair locally as a certificate signing request (.csr file), and then sent to their IT Security department to be signed and returned as a certificate (.cer file), which will then need to be added to a keystore.
- **Trusted Authority Signed Certificates:** The most secure level of authentication is a certificate that is signed by a trusted signing authority, issued specifically to the server on to which it will be installed. This certificate can be authenticated up the signature chain to a true trusted root authority. A certificate and keypair are created locally as a certificate signing request (.csr file), and then sent to a signing authority to be signed and returned as a certificate (.cer file), which will then need to be added to a keystore. This signing process is not free, and can take time to get, but offer the highest level of authentication. To use this type of certificate for internal servers like Voice Grid is

uncommon except for those organizations who are extremely concerned with security.

---

## A Little About Keystores

A keystore can contain many certificates and their private keys. Some organizations may put all certificates for their various servers into one keystore, while others may have a separate keystore per server. For the sake of simplification of the instructions below, we'll have a single certificate in the keystore, but that is not a requirement.

There are also multiple formats of keystores, such as JCEKS, JKS, PKCS12 and others, the most common being PKCS12. There are also means to convert between most keystore types, if necessary.

Most convenient and most-secure for Voice Grid use is to use Windows' internal, secure Certificate Repository. If a custom certificate is to be used, it can be imported directly into Windows, or it can be provided within its own keystore and imported into Windows from there. (It can also be used directly from that separate keystore without importing it to Windows, but that is not recommended because using a separate keystore will require the password to that keystore to be in clear-text within Voice Grid's `application.properties` file.)



To import the contents of a separate keystore file into Windows, a PKCS12 keystore is required.

---

## Using The Default CEP Client Certificate

The simplest and quickest way to get CEP up and running in Voice Grid is to use the certificate that the CEP server created when it was installed. Consult the CEP Server documentation for the location of that certificate, but it will be named `artemis.cer`.

---

## Windows Instructions

1. Copy the certificate file to anywhere on the Voice Grid server. The rest of these steps are all performed on the Voice Grid server.
2. Using Windows search in the task bar, type `mmc`, and then choose the "Run as administrator" option.
3. Once the MMC UI opens, in the top menu, click on `File`, then choose `Add/Remove Snap-in...`
4. When the Add or Remove Snap-ins dialog opens, click on the `Certificates` item at the left, then click on `Add >`.
5. When prompted for the type of account, choose `Computer account` then click `Next`.
6. When prompted for the computer to manage, choose `Local computer (the computer this console is running on)` then click `Finish`.

7. You should now see the Add or Remove Snap-ins dialog again. Click **OK** to return back to the main MMC window.
8. At the left, click on **Certificates**.
9. In the center pane will be a list of folders. RIGHT-click on the **Trusted Root Certification Authority** folder and then choose **All Tasks** then **Import**. This will open the Certificate Import Wizard.
10. Verify that the Store Location is set to **Local Machine**, then click **Next**.
11. Click on the **Browse...** button. This will open an "Open" file dialog.
12. Browse to and locate the artemis.cer file you copied from the CEP server.
13. Select the artemis.cer file, then click **Open**.
14. You will see a dialog that provides some information about the certificate. Click on **Next**.
15. Verify that the **Place all certificates in the following store** item is selected, and that the **Certificate store:** value is **Trusted Root Certification Authority**, then click **Next**.
16. Click **Finish**.
17. It should pop up a message saying "The import was successful." Click **OK**.
18. You should now be back at the main MMC window. In the center pane, DOUBLE-click on the **Trusted Root Certification Authorities** folder.
19. In the center pane, DOUBLE-click on the **Certificates** folder. The center pane should fill with a long list of certificates.
20. Find the certificate in the list that is named the same as the CEP server's name.
21. RIGHT-click on that certificate and choose **ScalingProperties**.
22. On the **General** tab, near the top, in the **Friendly Name** field, enter **CEP-Artemis** then click **OK**.
23. In the list of certificates, for that certificate, you should now see CEP-Artemis under the Friendly Name column. You may now close MMC. If it asks to save the Console settings, you may choose **No**.

---

## Linux Instructions



**You will need to re-perform this procedure after every Voice Grid update**, because the installer will replace Java's "cacerts" file with the one that comes with the JRE. This is necessary to ensure the JRE has all



the latest certificates, plus the CEP certificate you add.

**These instructions presume the use of the Azul Zulu Java Runtime Environment (JRE) that comes with Voice Grid.** If you are using your own JRE, then the paths in these instructions will not be accurate, and you will need to adapt the instructions to your JRE accordingly.

1. Copy the `artemis.cer` certificate somewhere on the Linux server, so it is accessible to Linux utilities:
2. Save a copy of the JRE's original "cacerts" truststore, then update the JRE's "cacerts" truststore.

```
$ cd /opt/Nexidia/VoiceGrid/zulu-jre/lib/security
$ sudo cp cacerts cacerts.orig
$ sudo ../../bin/keytool -import -trustcacerts -cacerts -storepass changeit -noprompt -alias CEP-Artemis -file
/path/to/where/you/saved/artemis.cer
```

3. **IMPORTANT: You should get a response stating "Certificate was added to keystore".**

4. Set ownership back to the nexidia user:

```
$ sudo chown nexidia:nexidia cacerts*
```

5. List the directory content. You should see something similar to this:

```
$ ls -l
total 492
-r--r--r-- 1 nexidia nexidia 2488 Feb 7 11:30 blocked.certs
-r--r--r-- 1 nexidia nexidia 127625 Feb 10 12:29 cacerts
-r--r--r-- 1 nexidia nexidia 126674 Feb 10 11:35 cacerts.orig
-r--r--r-- 1 nexidia nexidia 11770 Feb 7 11:30 default.policy
-r--r--r-- 1 nexidia nexidia 228507 Feb 7 11:30 public_suffix_list.dat
```

---

## Using Custom Certificates

---

### Importing Custom Trusted Root Certification Authority Certificates

Follow the steps under [Using The Default CEP Client Certificate](#) except specify your custom certificate instead of `artemis.cer`.

Windows Only: If your custom certificate is within a keystore file, at step 12, where you are browsing to the certificate, set the file type (to the right of the **File Name** field) to "All Files (.\*)" in order to see your keystore, and at step 13, select your keystore file and when you click on **Open** you will be prompted to enter the password to your keystore file.

---

## Importing a Dual-Use Certificate

If you have a single, custom certificate that is to be used for both encryption and authenticating the CEP server, see [Importing Custom Trusted Root Certification Authority Certificates](#), above, to install that single certificate.

---

## Importing Encryption Certificates

If using separate certificates for authentication and encryption, import the custom certificate to be used for authentication-only following the steps in [Importing Custom Trusted Root Certification Authority Certificates](#), above, and then install the certificate that is to be used for encryption-only following these steps:

1. Copy the encryption-only certificate file to anywhere on the Voice Grid server, or if that certificate is within a keystore, copy that keystore file to anywhere on the Voice Grid server. The rest of these steps are all performed on the Voice Grid server.
2. Using Windows search in the task bar, type `mmc`, and then choose the "Open file location" option. This will open a Windows Explorer window, usually in the `C:\Windows\System32` directory, and will highlight **mmc.exe**
3. While holding the SHIFT key, RIGHT-click on **mmc.exe**, then choose the "Run as different user" option.
4. When prompted for user credentials, enter the Voice Grid *service user's* credentials.
5. Once the MMC UI opens, in the top menu, click on **File**, then choose **Add/Remove Snap-in...**
6. When the Add or Remove Snap-ins dialog opens, click on the **Certificates** item at the left, then click on **Add >**.
7. Click **OK** to return back to the main MMC window.
8. At the left, click on **Certificates**.
9. In the center pane will be a list of folders. RIGHT-click on the **Personal** folder and then choose **All Tasks** then **Import**. This will open the Certificate Import Wizard.
10. Verify that the Store Location is set to **Current User**, then click **Next**.
11. Click on the **Browse...** button. This will open an "Open" file dialog.
12. Click on the drop-down to the right of the **File Name** field, and choose "All Files (.\*)"
13. Browse to and locate the custom certificate file or the keystore file containing the trusted certificate that file you copied from the CEP server.
14. Select that file, then click **Open**.
15. You will see a dialog that provides some information about the certificate. Click on **Next**.

16. Verify that the `Place all certificates in the following store` item is selected, and that the `Certificate store: value` is `Trusted Root Certification Authority`, then click `Next`.
17. You will be prompted to enter the keystore's password. Enter the password in the `Password` field, verify that the `Include all extended properties` option has a check-mark, then click `Next`.
18. Verify that the option selected is `Place all certificates in the following store` and the `Certificate Store` field contains "Personal", then click `Next`.
19. Click `Finish`.
20. It should pop up a message saying "The import was successful." Click `OK`.
21. You should now be back at the main MMC window. In the center pane, DOUBLE-click on the `Personal` folder.
22. In the center pane, DOUBLE-click on the `Certificates` folder. The center pane should fill list of personal certificates for the service user. (What you added may be the only one.)
23. Find the certificate in the list that is the one you added to use for Voice Grid to encrypt messages being sent to the CEP server.
24. RIGHT-click on that certificate and choose `ScalingProperties`.
25. On the `General` tab, near the top, you will see a `Friendly Name` field. This contains the alias to be used by Voice Grid to identify that certificate.
  - If there is already a value in the `Friendly Name` field, you may keep that, or you may change it to something else.
  - If there is nothing in the `Friendly Name` field , enter `CEP-Artemis` or any name of your choice.
  - Make note of that `Friendly Name`. You will need it later.
26. Click `OK`.
27. In the list of certificates, for that certificate, you should now see the pre-existing value or the value you entered under the `Friendly Name` column. You may now close MMC. If it asks to save the Console settings, you may choose `No`.
28. Refer to the [Setting CEP Properties In Voice Grid](#) section for instructions on how to set properties, and set the following two properties:
  - `cep.keystore.alias=TheFriendlyNameForTheCertificateYouJustAdded`
  - `cep.keystore.type=Windows-MY`

---

## Setting CEP Properties In Voice Grid

If using the default certificate provided by the CEP Server, installed exactly as directed in the [Using The Default CEP Client Certificate](#) section, configuration only requires configuring the mandatory CEP properties. Likewise, if a single custom certificate can be used to authenticate the CEP server and encrypt the data being sent to it. If the desire is to have one custom certificate that establishes the issuer of the custom certificates as a trusted authority, and another certificate that holds the encryption keys needed to communicate with the CEP server, then you **might** need to also set the `cep.keyStoreType` property as explained below.

Properties for CEP are set in the `<install-directory>\application.properties` file where `<install-directory>` is often `C:\Program Files\Nexidia\Voice Grid\NxVgMediaWorker`. That file can be edited with any text editor, such as Notepad, *but whatever editor is used, it should be launched using "Run as administrator" so that it has permissions to write to the file.*



**For all properties mentioned below, if there is a line in the file beginning with that configuration Key (property name), modify that line; however, unless CEP had been used previously, the lines for the properties will need to be added to the end of the file.**

Changes to property values will not be recognized until the Voice Grid service is restarted.

The format for all properties is:

`key=value`

and they are *case-sensitive*

---

### Mandatory Properties To Enable CEP

**`cep.broker.hostFQDN=FQDN-of-the-CEP-server`**

Setting this property enables CEP and tells Voice Grid the CEP server's Fully Qualified Domain Name.

Example:

`cep.broker.hostFQDN=myCEPserver.somesubdomain.company.com`

---

### Optional CEP Properties

The values shown below are the default values used when the property is *not* specified in the `application.properties` file.

**`cep.broker.port=30101`**

---

Only if your CEP server uses a custom port number rather than the default port of 30101, use this property to specify that custom port number.

---

#### **cep.queue.exceptions=VoiceGrid.Exceptions**

---

This specifies the queue name on which the CEP server expects to receive exceptions from Voice Grid. Needing to set this is very uncommon.

---

#### **cep.queue.heartbeat=Heartbeats**

---

This specifies the queue name on which the CEP server expects to receive heartbeat messages from Voice Grid. Needing to set this is very uncommon.

---

#### **cep.heartbeatSeconds=60**

---

This specifies how often Voice Grid sends heartbeat messages to the CEP server. Any change here will likely need to be coordinated with CEP server settings that indicate how often to expect heartbeats. Needing to set this is very uncommon. Setting this to 0 will disable heartbeats.

---

#### **cep.exceptionBacklogMaximum=200000**

---

This specifies the maximum number of exception messages that Voice Grid should queue up to send to the CEP Server if the CEP server is down / unreachable. The messages will be sent in order once a connection is (re-)established. If there are more exceptions than this value during the time that Voice Grid cannot communicate with the CEP server, the oldest messages will be discarded as new ones are added.

---

#### **cep.verifyHost=true**

---

This specifies whether to verify the CEP server's hostname. If custom certificates are used that do not include the ability to verify the CEP server's FQDN (Fully Qualified Domain Name), OR if DNS lookup is disabled on the Voice Grid server, preventing such verification, then this property can be set to false (lower-case) to disable verification in order to allow Voice Grid to permit communication with the CEP server without verification. Needing to set this is very uncommon and should be avoided unless absolutely necessary.

---



Warning! Setting `cep.verifyHost` to false reduces the level of security by permitting some other, unknown entity to pose as the CEP server, allowing potentially-sensitive data to be acquired.

---

### `cep.keystore.alias=CEP-Artemis`

---

This specifies the alias or Friendly Name used to find the encryption certificate in the keystore. (See [Using The Default CEP Client Certificate](#) and [Using Custom Certificates](#).) Needing to set this is very uncommon.

---

### `cep.keystore.type=Windows-ROOT`

---

This specifies where the certificate used for encrypting messages to the CEP server is located. If custom certificates are being used and there are separate certificates for providing the encryption keys and providing a trusted signing authority used to validate the encryption certificate, AND if you do not wish to add the encryption certificate as a trusted root authority, then the encryption certificate can be installed into the Personal certificates for the service user (see [Using Custom Certificates](#)), in which case this will need to be set to `Windows-MY`.

There are other values that are allowed for special-use cases for quickly testing certificates without importing them into windows. (See [Properties For Special-Use Test Cases](#).)

---

## Properties For Special-Use Test Cases

When using custom certificates, it may be desirable to test those certificates (especially when testing new certificates that will replace soon-to-expire certificates) quickly, without having to import them in to Windows. **These properties are not intended to be used other than for short-term testing, as they expose their test keystore (and possibly truststore) passwords.**

To test, these value would be set in the `application.properties` during testing, and then deleted from `application.properties` after testing, or in the case of `cep.keyStoreAlias`, `cep.keyStoreType` and `cep.trustStoreType`, set back to their original values if they had been previously set.

- To test a new encryption certificate, that certificate can be placed in an external keystore file. How to do that goes beyond the scope of this document, but acceptable keystore types are JKS, JCEKS, and the most common being PKCS12.
- To test a new Trusted Root Certification Authority certificate, such as if the encryption certificate being tested requires authority other than one already imported into Windows, or if you need to test a new Trusted Root Certification Authority certificate before replacing an expiring one, that certificate can be placed into an external keystore (referred to as a truststore). How to do that goes beyond the scope of this document, but acceptable keystore types are JKS, JCEKS, and the most common being PKCS12.

- If there is only one certificate to be used for both encryption and certification, then that keystore can be specified as both the keystore and truststore, but unless you intend to use a Trusted Root Certification Authority certificate that has already been imported into windows, you must set both the keystore and truststore properties, below.

## Specifying An External Keystore File

### **cep.keystore.path**

---

Set this to the full path and file name of your test keystore. Do NOT use windows backslashes (\) in the path, and instead use forward slashes (/).

---

### **cep.keystore.password**

---

Set this to the password for the test keystore. This would have been set when you created the keystore.

---

### **cep.keystore.type**

---

Set this to the type of the test keystore — JKS, JCEKS or PKCS12. If not set, but cep.keystore.path is set, this will default to PKCS12

---

### **cep.keystore.alias**

---

Set this to the alias for the encryption certificate contained in the keystore

---

## Specifying An External Truststore File

### **cep.truststore.path**

---

Set this to the full path and file name of your test truststore. Do NOT use windows backslashes (\) in the path, and instead use forward slashes (/).

---

### **cep.truststore.password**

---

Set this to the password for the test truststore. This would have been set when you created the truststore.

---

### **cep.truststore.type**

---

---

Set this to the type of the test truststore — JKS, JCEKS or PKCS12. If not set, but cep.truststore.path is set, this will default to PKCS12

---